

全国大学生计算机系统能力大赛操作系统赛
决赛产品文档

AgentOS-uCore

系统原生智能体运行与治理平台

在 RISC-V uCore 中实现可信身份、上下文、工具、调度、证据与多智能体协作

作品名称: AgentOS-uCore / AgentOS Nexus
团队: AgentOS-uCore 团队
目标平台: RISC-V 64 / uCore / QEMU
文档性质: 决赛完整产品与工程验证说明

本文只陈述当前仓库中已经实现、能够定位并具有验证入口的能力。

阅读提示

本文把四种证据严格分开：静态合同说明接口和边界存在；cross build 说明内核与 Guest 能以目标 ABI 编译链接；QEMU Guest 场景说明实际运行路径成立；配对性能数据说明给定负载、样本和口径下的观察结果。任何一种证据都不替代另外三种。

文中“模型调用”只在真实 provider 或同路径 replay 实际完成时成立；“性能改善”只引用冻结 one-shot 数据或明确的竞赛基准，不从日志热闹程度、observer 刷新速度或静态代码形状反推。

诚实边界

内核负责可信身份、能力、scope、Context、调度、等待、资源和证据；Guest 负责规划、历史、委派和工具选择；Host 只负责终端、本地路由、TLS、API key 与供应商协议转换。JSON、HTTP、TLS、模型 SDK 与 MCP wire 均由 Guest/Host 层处理。

摘要与目标-证据矩阵

摘要

传统操作系统能够创建进程、分配内存和调度线程，却不能直接表达智能体工作流程中的角色身份、跨轮上下文、工具权限、数据来源、任务等待和资源归属。应用框架可以约定这些概念，但一旦用户态组件出错或越权，约定本身不能成为强制边界。我们据此在 RISC-V uCore 上实现 AgentOS-uCore：让内核识别 Agent 与 workflow，把可信身份、Context Path、结构化工具调用、文件元数据查询、事件等待、资源记账、workflow 调度与可验证证据连接成一条可运行路径。

我们的核心选择是“机制进入内核，策略留在用户态”。内核不理解自然语言，也不替模型决定工具；它在每次工具、消息、文件访问和生命周期变化处检查 identity、capability、scope、schema 与 provenance。Guest 用户态可以采用确定性策略、真实模型或 replay，但必须沿相同系统调用和 Context 路径取得事实。Host 只承担 QEMU 串口、TLS、API key 与 provider JSON 转换，不能读取 Guest 业务文件或伪造工具结果。

在基础机制之上，我们交付两个互补产品面。contest-demo 使用确定性科研 workflows 和等量 AB/BA 负载，保证现场性能比较可重复；AgentOS Nexus 则提供长期运行的双窗口交互体验。评委在左侧终端连续提交自然语言目标，Coordinator 根据真实返回动态委派 System、Research 和 Analyst，右侧 observer 同步显示内核实际允许、拒绝、等待、唤醒和记录了什么。四个业务 Agent 都拥有独立 PID、Agent identity、Context、role/capability。四个角色共享 workflow lifecycle、scope、资源域和调度实体；跨 Agent 大数据写入带 SHA-256 和 provenance 的 Guest 工件，任务消息只传 canonical 控制包与 generation-safe handle。

我们建立了从模型、Guest、内核到 Host 的严格验证边界。固定 replay 走与 live provider 相同的 QEMU、串口、TASK/MESSAGE、typed V2、artifact 和 Context 路径，但只证明 fixture 绑定的闭环，不冒充云模型质量。性能部分引用冻结 one-shot campaign：30 次 fresh QEMU boot、33 个原始文件、19 张长表和 7,498 行逐样本记录。16 个独立 workflow 配对中 indexed core path 全部更快，配对加速比中位数为 3.118 倍；文件查询完整网格包含 $3 \times 4 \times 15 = 180$ 个配对；任务传输包含 96 条 16-op sequence；504 条 exact EEVDF wake probe 全部落在 0-1 tick。我们同时保留端到端差值、顺序效应、测量窗口和独立单位等限制，不把 core-path 观察扩大为所有场景的系统级加速。

作品定位

AgentOS-uCore 将内核强制机制、Guest 策略与 QEMU 复验连接成完整的 Agent workflow 系统；AgentOS Nexus 是这些机制的交互式纵向集成产品。

主要贡献

1. 我们建立可信 Agent identity、role/capability、workflow id+generation 与 VFS scope，使普通进程和 Agent 在同一内核中共存而不混淆授权主体。
2. 我们实现 25 项内核工具目录、exploratory typed V2、ENFORCE V3 冻结执行合同、顺序 compact batch 和有界 Task SQ/CQ core，明确区分各自保证。
3. 我们把七页 Context 地址区、因果路径、六类 provenance 标签、Evidence Ring 和 challenge-bound fence 连接起来，使工具结果既能反馈给模型，又不能自动提升为可信控制指令。
4. 我们实现显式 boot-scoped 文件 metadata、选择性索引、typed live query、generation resync，并用同一负载的 scan/index 路径测量工作量和延迟。
5. 我们按 workflow 而非线程聚合资源与调度实体，以 U/P/F Credit Domain、Tool Phase Lease 和 EEVDF 管理峰值、服务量、睡眠与唤醒。
6. 我们交付确定性竞赛基准、长驻控制台和四业务 Agent Nexus，使身份、工具、Context、文件、等待、审批、工件和观测在同一实际 QEMU boot 中闭环。

目标-证据矩阵

表 1: 赛题目标、生产实现与证据边界

任务	约束	我们的实现	主要验证	结论边界
一	Agent 创建、Context 可用、普通进程共存	可信映像与角色发布；七页 Context；workflow lifecycle；exec/storage 账户	agentfinal_ucore、agenttrust_ucore、scope/lifecycle mutation	单 Hart；生命周期和槽位有界
二	至少三项结构化工具、发现与错误处理	25 项目录；V2 typed KV；ENFORCE V3；64-op compact batch；16-slot Task core	UAPI/layout checker、agenttoolabi_ucore、agentcontract_ucore、agenttask_ucore	V2/batch 不具有冻结 DAG 保证
三	至少五轮调用、直接访问、超长淘汰	128 条可信 Context Path；六页只读 mirror 加一页 user cache；因果、rollback、provenance	agentfinal_ucore 的 64 次顺序调用、snapshot/rollback/eviction	rollback 不逆转已经发生的外部效果
四	至少两类文件扩展与查询性能比较	显式 metadata、summary/digest、status/stage/kind 索引、typed watch、resync	agentfs_ucore、agentbench_ucore、180 个 one-shot file-query 配对	仅索引当前 boot 显式登记对象

任务	约束	我们的实现	主要验证	结论边界
五	至少两类 Loop 机制、多 Agent 稳定运行	watch/wait、heartbeat、可信 route、LLM correlation、workflow EEVDF、证据环	agentloop_ucore、agent_eevdf_ucore、504 条 exact wake probe	内核不规划、不调用模型
六	三模块以上综合、QEMU 程序、性能对比	确定性 labdemo_ucore；plain/AgentOS 双目标；交互控制台；四角色 Nexus	contest-demo、双目标 runner、Nexus strict replay 与 live 显式入口	Replay 不等于 live；Nexus 不替代性能基准

核心实测快照

在源码提交 2b14fb1f74b9bd093e6de939a16554620835699e 上，冻结数据包含 30 次 fresh QEMU boot、33 个 raw 文件、19 张长表和 7,498 行记录。本文后续所有性能图表都从该规范运行的 CSV 读取，不从聚合数字反推原始样本。

目录

摘要与目标-证据矩阵	iii
第一章 问题价值与作品边界	1
1.1 从进程抽象到智能体工作流	1
1.2 需要进入内核的机制	1
1.3 增量边界：保留 uCore 的教学可读性	1
1.4 三类工具执行路径为何必须分开	2
1.5 从参考思想到本项目增量	3
1.6 产品价值：把“Agent 想做什么”和“系统允许什么”放在同一现场	3
1.7 本章验证落点	3
第二章 总体架构与端到端运行流程	4
2.1 四层结构	4
2.2 一次用户回合的闭环	5
2.3 同一工作流的机制对照	6
2.4 源码落点	6
第三章 任务一：可信 Agent 身份、Context 与生命周期	9
3.1 约束	9
3.2 设计：身份不是用户态标签	9
3.3 七页 Context 地址区	9
3.4 Workflow lifecycle 与三类 gate	10
3.5 资源账户随 workflow 一起创建	10
3.6 实现落点	11
3.7 验证	11
第四章 任务二：结构化工具、执行合同与 Task Channel	12
4.1 约束	12
4.2 工具发现和模型呈现分层	12
4.3 Exploratory typed V2	12
4.4 Compact batch	13
4.5 ENFORCE V3 冻结执行合同	13
4.6 Tool Phase Credit Lease	13
4.7 Typed Task Channel core	13

4.8	验证	14
第五章	任务三：Context Path、Provenance 与可验证证据	15
5.1	约束	15
5.2	Context 记录与因果路径	15
5.3	Rollback 的精确语义	15
5.4	六类固定 provenance 标签	16
5.5	Manifest 与 pre-effect gate	16
5.6	Fence-Sealed Evidence Ring	16
5.7	Workflow fence 的精确 cut	17
5.8	验证	17
第六章	任务四：Agent Live-Query FS 与文件查询	18
6.1	约束	18
6.2	显式 metadata 与真实 inode 绑定	18
6.3	Scan、选择性索引与结构化结果	18
6.4	Typed live watch	19
6.5	增量丢失与 resync	19
6.6	文件查询 one-shot 实验	19
6.7	实现与验证	20
第七章	任务五：Agent Loop substrate 与 Workflow EEVDF	21
7.1	约束	21
7.2	有界事件队列与 watch/wait	21
7.3	Heartbeat 是事件，不是内核业务	21
7.4	绑定身份的 LLM correlation RPC	21
7.5	Workflow Credit Domain	22
7.6	按 workflow 聚合的 EEVDF	22
7.7	调度实测	22
7.8	实现与验证	23
第八章	任务六：确定性综合场景与可重复基准	24
8.1	约束	24
8.2	确定性科研恢复工作流	24
8.3	Native/Compat 与 AB/BA	24
8.4	冻结 one-shot 主比较	25
8.5	工作量与 I/O 口径	25
8.6	与 Nexus 的职责分离	26
8.7	验证	26

第九章 AgentOS Nexus：系统原生智能体协作中枢	27
9.1 从单次循环到长驻工作台	27
9.1.1 约束	27
9.1.2 设计	27
9.1.3 实现	27
9.1.4 验证	27
9.2 N1 通用子任务协议	28
9.2.1 约束	28
9.2.2 设计与实现	28
9.3 Artifact 数据平面	29
9.3.1 约束	29
9.3.2 设计	29
9.3.3 黄金场景的真实数据依赖	29
9.4 动态工具呈现与模型回路	29
9.4.1 工具目录不等于 prompt	29
9.4.2 真正的观察后再决策	30
9.5 调用级审批、取消与安全降级	30
9.5.1 参数绑定审批	30
9.5.2 取消和关闭	30
9.6 独立内核观测面	31
9.7 交互控制面	31
第十章 跨模块安全与调度治理	32
10.1 安全不可约化为一张 allowlist	32
10.1.1 约束	32
10.1.2 授权链	32
10.2 Provenance 作为流转属性	32
10.2.1 设计	32
10.2.2 威胁与对策	33
10.3 U/P/F 资源账户	33
10.3.1 约束	33
10.3.2 设计	34
10.4 Phase Lease 与安全包络	34
10.5 Workflow EEVDF 与防线程放大	34
10.5.1 约束	34
10.5.2 设计与实现	34
10.6 Evidence Ring 与 Workflow Fence	35
10.7 安全与可用性的取舍	35

第十一章 功能、可靠性与性能验证	36
11.1 证据分层	36
11.2 功能验证矩阵	36
11.3 可靠性与负路径	37
11.3.1 静态合同与 mutation	37
11.3.2 构建与栈安全	37
11.3.3 QEMU 负路径	38
11.4 Nexus 严格 replay 验收	38
11.4.1 方法	38
11.4.2 黄金路径	38
11.5 一次性性能数据集	39
11.5.1 数据来源与独立单位	39
11.6 Core 与端到端窗口	39
11.7 File query catalog 工作量	39
11.8 Task 通道序列分布	40
11.9 EEVDF 唤醒与公平性	40
11.10 结果解释的最小规则	40
第十二章 已知限制与适用范围	41
12.1 产品能力边界	41
12.1.1 Nexus 不是任意多 Agent 平台	41
12.2 N1 协议边界	41
12.3 Artifact store 边界	41
12.4 会话与模型边界	42
12.5 观测边界	42
12.6 审批、取消与回滚	42
12.7 V2、V3 与 Task Channel 不同保证	43
12.8 MCP/A2A 与外部资源	43
12.9 内核和文件系统边界	43
12.10 性能证据的外推边界	43
第十三章 工程组织、交付与演进	45
13.1 增量工程原则	45
13.2 仓库组织	45
13.3 四类产品入口	46
13.4 构建与回归	46
13.4.1 建议顺序	46
13.4.2 共享输出约束	47

13.5 交付物和可追溯性	47
13.6 从确定性流程到 Nexus 的工程演进	47
13.7 后续演进路径	48
第十四章 总结	49
附录 A 关键源码与证据索引	50
A.1 内核身份、生命周期与 Context	50
A.2 工具、合同与 Task Channel	50
A.3 Context、Provenance、Evidence 与 Fence	51
A.4 文件查询、索引与 Live Watch	52
A.5 等待、IPC、资源与 EEVDF	52
A.6 综合工作流与 Nexus	53
A.7 性能数据和图表	54
附录 B 复验指南与附加数据解释	56
B.1 环境准备	56
B.2 分层构建	56
B.3 Nexus 产品复验	56
B.3.1 静态合同	56
B.3.2 固定 replay 闭环	57
B.3.3 现场交互	57
B.4 Contest demo 与 paired 证据	57
B.5 一次性数据集的离线检查	58
B.6 附加 I/O 混合 scope 图	58
B.7 核心数字复核表	59
B.8 报告工程的重建	59

插图

2.1	AgentOS-uCore 总体架构：Host 传输、Guest 策略与内核治理分离	4
2.2	Nexus 单回合“目标-模型-委派-工具-观察-再决策”闭环	6
2.3	同一业务 workflow 在 Plain uCore 与 AgentOS-uCore 上的系统责任对照	8
6.1	Catalog size 与 exact hit count 的实测 scan/index 加速比；每格 15 个 inner pair	19
6.2	显式 warmup 后的 first/repeat scan 与 ready index 分组	20
7.1	EEVDF exact wake latency 与并发 1-4 的 Jain fairness	23
8.1	Core-path 与端到端窗口的配对结果及顺序分层	25
8.2	16 个独立 QEMU boot 的 traversal/indexed core 配对原始分布	26
11.1	4 个 boot、8 个旋转顺序下 96 个 16-op sequence 延迟分布	40
B.1	内核 I/O 和工作量归一化热力图；owner/window/scope 不同，仅作附加工程视图	58

表格

1	赛题目标、生产实现与证据边界	iv
1.1	机制与策略的责任分界	2
1.2	工具执行路径的用途与保证	2
2.1	总体架构的运行实体	5
2.2	总体架构的关键生产源码	6
3.1	Agent 身份字段及安全用途	10
5.1	Context provenance 词汇	16
9.1	Nexus 四个业务角色的真实系统身份	28
10.1	当前系统中主要威胁与 fail-closed 对策	33
11.1	AgentOS-uCore 的六层验证证据	36
11.2	六项任务的关键功能与证据入口	36
11.3	一次性采集的实验族与正确独立单位	39
13.1	主要代码目录与责任	45
A.1	任务一与 Context 主路径	50
A.2	任务二主路径	51
A.3	任务三与截面证据主路径	51
A.4	任务四主路径	52
A.5	任务五与跨模块调度主路径	53
A.6	任务六和交互产品主路径	53
A.7	性能证据主路径	54
B.1	正文引用的核心数字及其正确解释	59

问题价值与作品边界

1.1 从进程抽象到智能体 workflow

经典进程抽象回答“谁在执行、使用了多少 CPU 和内存、何时阻塞”，但智能体 workflow 还要持续回答四个问题：当前决策属于哪个角色和 workflow；它依据了哪一轮上下文；工具副作用是否在授权范围内；跨 Agent 结果是可信控制还是不可信数据。若这些信息只存在于 prompt、JSON 或应用日志中，内核无法在真正发生文件、IPC 和资源效果之前检查它们。

我们不试图让操作系统代替模型思考。我们把问题重新表述为：操作系统能否提供一组足够小、能够强制、可观测又能被不同策略复用的机制，让用户态 Agent Loop 在失败、并发和攻击输入下仍保持清晰边界。这个表述避免了两个极端。一端是把现有脚本换成“Agent”名字却仍然硬编码所有结果；另一端是把模型 SDK、网络协议和自然语言解析塞进教学内核，扩大可信计算基并破坏可验证性。

我们的价值判断

我们以可重验的系统属性评价 AgentOS：用户态智能体完成真实任务时，身份、权限、资源、因果和证据都能由内核独立检查。

1.2 需要进入内核的机制

我们按“效果发生点”选择内核增量。进程成为 Agent 时，内核发布 identity、role、control id 和 capability；进入 workflow 时，内核绑定不可混淆的 id+generation 与 VFS scope；工具调用时，内核解析版本化参数并检查 schema、capability 和数据来源；消息与文件结果进入下一轮前，内核保守传播 provenance；等待模型或事件时，内核使线程真实休眠；调度与资源效果则按 workflow 聚合，而不是让一个 workflow 通过增加线程获得额外公平份额。

表 1.1 汇总可信边界。我们刻意让用户态保持丰富、内核接口保持紧凑。这样既能使用真实模型，也能用 deterministic replay 或固定策略验证同一底层路径。

1.3 增量边界：保留 uCore 的教学可读性

作品建立在 LearningOS/uCore 教学内核之上[1]。我们保留原有进程、页表、VFS、trap、调度和系统调用主干，只在明确边界增加 Agent 字段、workflow lifecycle、Context 映射、工具协议、元数据、调度账户和观测设施。普通进程仍可运行；Agent 身份只能由受控 create/exec

表 1.1: 机制与策略的责任分界

层次	负责内容	明确不负责
AgentOS 内核	身份、role/capability、scope、Context、provenance、typed ABI、route/wait、资源、调度、审计和 fence	自然语言规划、prompt 内容判断、工具选择、模型推理、JSON/HTTP/TLS/MCP wire
Guest 用户态	目标分解、角色策略、历史、工具呈现、TASK 状态机、artifact 组织、错误重规划	绕过内核 capability、伪造 Context 因果、替换内核资源账本
Host	CLI、observer、本地 socket、QEMU 串口、API key、TLS、provider 协议转换	读取 Guest 业务文件、选择业务工具、代写工具结果、创建内核证据

路径发布。所有新增 UAPI 使用定长结构、显式 version 和 size，并以 kernel/user 共用头文件及 layout checker 固定 ABI。

我们没有修改文件系统磁盘格式。Agent metadata 是当前启动周期内显式登记的内存目录，并绑定真实 inode incarnation；live query 只索引这些对象。我们也没有扩大 uCore 的 MAXFILE 来容纳产品程序，而是在构建期约束 Nexus Guest raw binary 不超过 274432 字节。这个取舍使新增能力仍受原教学内核资源边界约束。

1.4 三类工具执行路径为何必须分开

作品同时保留 compact batch、typed V2、ENFORCE V3 与 Task SQ/CQ，分别服务同步操作、探索式调用、冻结合同和有界提交/完成。

表 1.2: 工具执行路径的用途与保证

路径	输入形态	适用场景	边界
Compact batch	最多 64 个 agent_op	固定顺序、低入口调用次数的同步操作	不是 typed KV、并行或 non-blocking batch
Typed V2	uint64/string KV	模型依据上一结果探索式选择下一工具	逐次检查，但没有冻结 DAG 的 predecessor/attempt 包络
ENFORCE V3	V2 prefix 加 contract/node/attempt/manifest	预先声明的高保证 DAG	动态 Agent Loop 不能假装调用图已经冻结
Task SQ/CQ	16-slot single-issuer ring	有界提交/完成、backpressure、resync 研究	当前同步 null/NONE provider，无业务 payload backend

Nexus 选择 V2，使 Coordinator 能根据刚返回的 System 或 Research 工件改变下一步；高保证固定流程使用 V3。Task Channel 当前范围止于 null/NONE provider 和有界 SQ/CQ 语义，业务 payload backend 留待后续实现。

1.5 从参考思想到本项目增量

我们参考 Linux `cpuacct/rstat` 的批量资源记账思想 [2, 3]、BPF ring buffer 的有序提交思想 [4]、Linux `EEVDF` 的 lag 与 virtual deadline 概念 [5]、`io_uring` 的 SQ/CQ 通信模型 [6]、Haiku BFS 的属性和选择性索引 [7]。在 Agent 方向，我们参考 tool-use 循环中“模型选择、应用执行、结果回灌”的职责划分 [8, 9]，并研究 MCP/A2A 的对象形状 [10, 11]。

这些工作提供概念来源；兼容范围以本项目已经实现的协议和测试为准。我们的增量是把相关思想约束到 uCore 的 workflow identity、Context、capability、provenance、VFS 和 evidence 语义中。Credit Domain 为同一 workflow 的 exec/storage 账户建立 U/P/F 状态和 fence 精确结算；Evidence Ring 密封 Agent Context、gap、credit 和 metadata generation；Task Channel 当前实现 AgentOS 自有的有界 SQ/CQ ABI。

原创与互操作边界

仓库没有 vendoring Linux、Haiku、AIOS、相关论文原型、MCP/A2A SDK 或 Wasm runtime 源码。MCP/A2A 当前是 deterministic in-memory 对象映射研究，不是 server、streaming transport、内核 adapter 或跨实现互操作证明。

1.6 产品价值：把“Agent 想做什么”和“系统允许什么”放在同一现场

确定性 `contest-demo` 解决性能复验问题，AgentOS Nexus 解决产品交互和机制可见性问题。前者保持同一业务合同和 AB/BA 次序，不让模型随机性污染对比；后者让评委临时提出目标，观察模型如何委派，又能在第二窗口按 task/correlation 对齐内核 route、WAITING、wake、capability、Context 与拒绝原因。

这种双产品面比单一 Dashboard 更有解释力。左侧结果说明策略确实消费了工具和工件；右侧观测说明这些动作没有绕过操作系统。两者共同展示“用户态可自由演进，内核边界保持可重验”的系统价值。

1.7 本章验证落点

本章边界由以下仓库事实交叉约束：公开 UAPI layout checker 固定结构尺寸和偏移；模块 checker 限制职责归属；Guest 场景验证系统调用行为；Host tests 验证 frame、sequence、hash 和 fail-closed；one-shot campaign 冻结性能原始数据。第十一章给出完整分层验证矩阵，第十二章集中列出我们没有交付的能力。

总体架构与端到端运行流程

2.1 四层结构

AgentOS-uCore 的总体结构分为交互、Host 基础设施、Guest 策略与数据、内核机制四层。四层直接对应权限和证据边界。自然语言只在交互、provider 与 Guest 策略层出现；真正产生文件、IPC、资源和调度效果的调用必须穿过内核机制层。

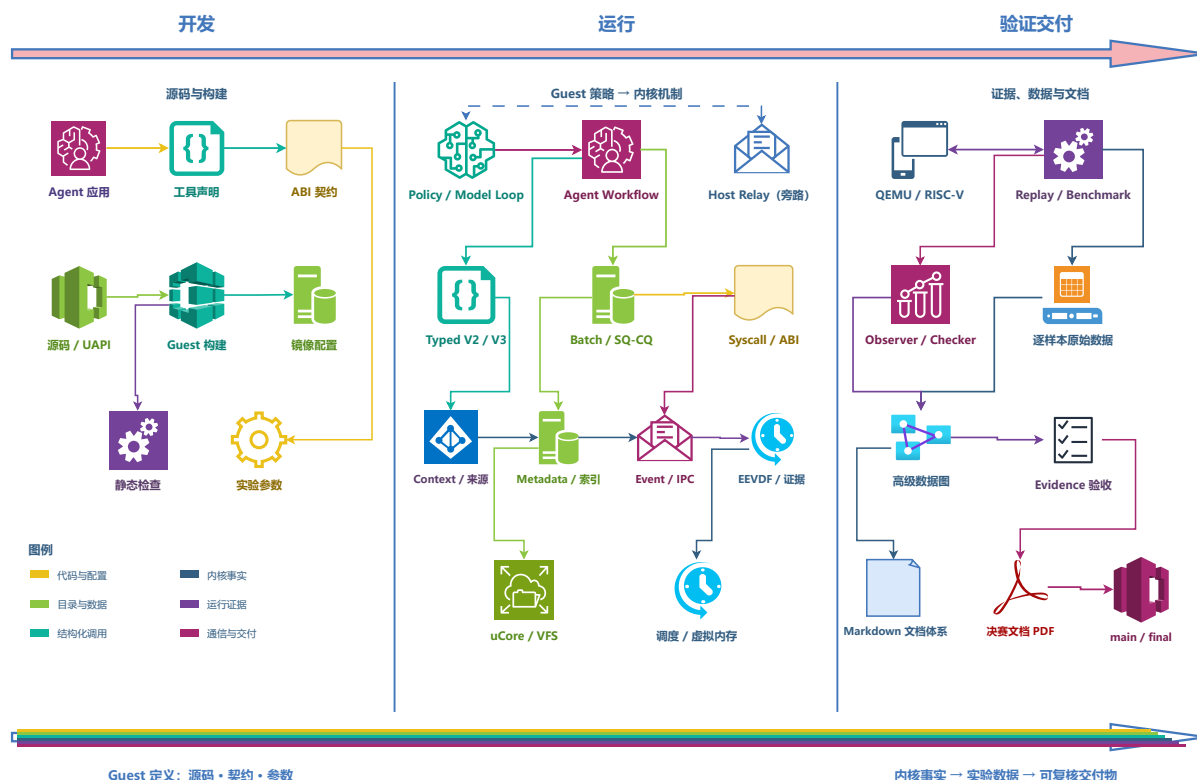


图 2.1: AgentOS-uCore 总体架构: Host 传输、Guest 策略与内核治理分离

Host daemon 是串口和本地 socket 的唯一 owner。一个 controller 可以提交输入与审批，多个 observer 只能读取事件。Guest Relay 是同一 workflow 中的基础设施 Agent，负责把模型请求和响应转换成有界 frame，但不计入 Nexus 四个业务角色，也不选择工具。Coordinator 是业务根；System、Research、Analyst 长驻并拥有独立 PID、Agent id、control id、role/capability 与 Context，同时共享 workflow lifecycle、scope、资源域和调度实体。

启动时，受信映像 manifest 约束可创建角色，内核为各业务进程分配 identity、control id、capability 与 Context，并为 workflow 建立共享资源账户；Guest 再安装 route/watch 并登记本 boot seed。业务面包含 Coordinator、System、Research、Analyst 四个角色，Transport Relay 只承担基础设施传输。这些步骤全部完成后才发布 Nexus profile 和 ready marker，避

表 2.1: 总体架构的运行实体

层	实体	输入/输出	可信职责
交互层	Nexus CLI、Observer	用户消息、控制、审批、只读 telemetry	只呈现和收集，不产生业务事实
Host 层	Console daemon、DeepSeek/Replay adapter、QEMU serial	本地消息、完整性 frame、provider JSON	持有 key/TLS；不访问 Guest 业务 VFS
Guest 层	Relay、Coordinator、System、Research、Analyst、Artifact Store	模型 wire、N1 TASK、typed V2、handle	规划、策略、工件和状态机
Kernel 层	identity、Context、tool、VFS、IPC、resource、scheduler、evidence	系统调用与可信状态	强制身份、权限、因果、资源和审计

免把普通会话误呈现为系统原生多 Agent 会话。具体身份发布和协议校验在第 九章展开。

2.2 一次用户回合的闭环

图 2.2展示一次目标如何从自然语言进入内核工具，又如何把真实结果送回模型。关键不在箭头数量，而在每一轮 tool_result 都来自 Guest 经过内核执行、Context 记录和工件重验后的事实。

- Host 将新的用户消息作为 turn 输入交给 Coordinator。
- Coordinator 结合有界历史和当前可见工具，生成模型请求后进入内核等待。
- Relay 通过完整性帧把请求交给 live provider 或同路径 replay，返回事件唤醒 Coordinator。
- Coordinator 验证模型动作；需要专门事实时，将 typed TASK 委派给 System、Research 或 Analyst。
- Worker 使用自己的 identity、Context、capability 与 workflow 共享资源账户执行 role-filtered V2 工具，等待期间不忙轮询。
- 短状态经 MESSAGE 返回，长结果进入 workflow-scoped artifact store；Coordinator 重验身份、状态迁移、handle 和 digest。
- 稳定、决策相关的结果投影回灌模型。失败和审批拒绝同样作为结构化观察，使模型可重新规划。
- 模型返回 final 时只完成当前 turn，会话、QEMU 与四个业务 Agent 继续长驻。

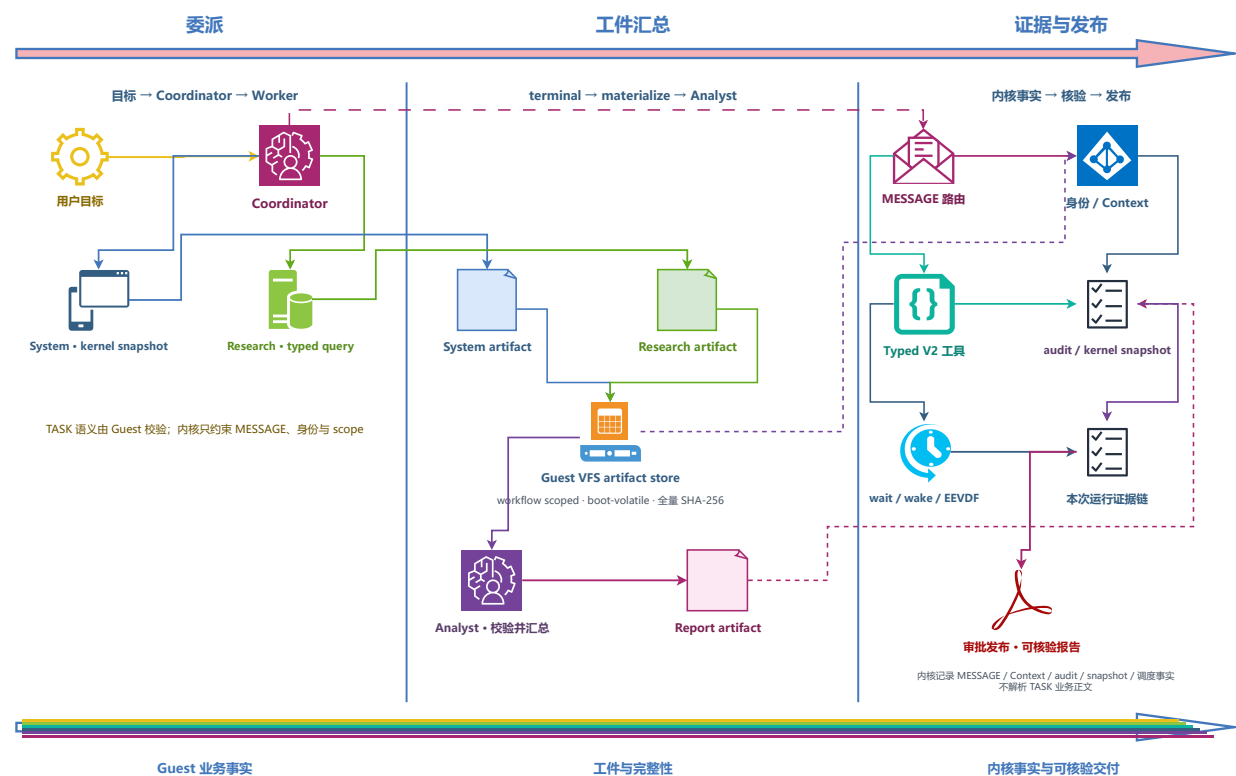


图 2.2: Nexus 单回合“目标-模型-委派-工具-观察-再决策”闭环

2.3 同一工作流的机制对照

图2.3两侧保持相同的业务输入、阶段和结果合同。Plain uCore 依靠普通进程、文件与 IPC 完成工作；AgentOS-uCore 在同一工作流下增加 typed ABI、可信身份与 Context、索引与 Live Watch、受治理 IPC、workflow EEVDF、Evidence Ring 和 Workflow Fence。对照在相同业务负载下聚焦系统责任的增量。

Nexus 进一步把交互划分为控制、artifact 数据和内核证据三类平面；三者的 N1、审批、provenance 与 observer 对齐规则集中在第9章说明。

2.4 源码落点

表 2.2: 总体架构的关键生产源码

组件	源码	职责
Host session/daemon	host_tools/agentos_relayd.py	本地路由、frame、provider、审批与 telemetry allowlist
CLI/observer	host_tools/agentos_cli.py; host_tools/agentos_observe.py	交互呈现和只读观测

组件	源码	职责
Provider adapter	host_tools/guest_llm_relay.py	provider 协议翻译
Nexus Guest	user/src/agentnexus_ucore.c	Relay、Coordinator、specialists、model loop 与 observer
N1/Artifact ABI	user/include/agent_nexus_protocol.h	TASK wire、角色、handle、manifest 与 capsule
Nexus shared runtime	user/lib/agent_nexus.c	发现、typed call、identity registry、artifact publish/read
Identity/role	os/agent_identity.c; os/agent_core.c	role policy、control、Agent 创建与工具 dispatch
IPC/wait	os/agent_ipc.c	route、watch、MESSAGE、heartbeat 与非忙等
Context/provenance	os/agent_context.c; os/agent_provenance.c	因果记录与标签传播
Observer/scheduler	os/agent_observe_*; os/workflow_scheduler.c	audit/timeline 和 workflow
		EEVDF

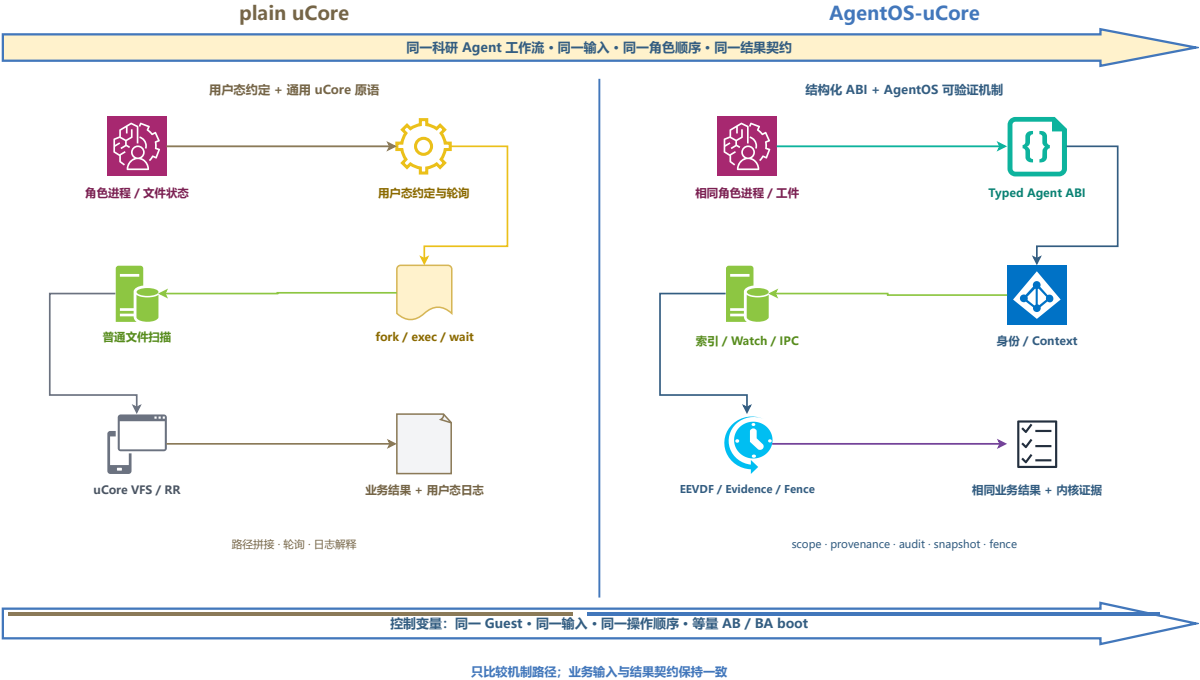


图 2.3: 同一业务 workflow 在 Plain uCore 与 AgentOS-uCore 上的系统责任对照

任务一：可信 Agent 身份、Context 与生命周期

3.1 约束

任务一要求操作系统能够创建 Agent，为它提供专用 Context 区，同时保持普通进程与 Agent 进程共存。真正的难点不是在 PCB 增加一个布尔值，而是确保身份只能由可信路径发布、fork/exec/exit 不泄漏权限、旧 workflow 的 PID 或 slot 不能在复用后命中新对象，并让 Context “可直接读写”的要求不变成伪造授权历史的入口。

我们把 workflow admission、初始身份、Context 和资源账户作为同一个创建事务处理。任何一步失败都撤销已经发布的字段和页面；成功后，后续工具、VFS、IPC、调度和审计都只读取内核中的同一身份事实。

3.2 设计：身份不是用户态标签

Agent 身份由 `is_agent`、`agent_id`、`kernel role`、`control id`、`controller id`、`capability mask` 和 `workflow lifecycle` 共同构成。`role policy` 在内核中固定映射能力与调度权重。例如 Sentinel 具有 `metadata/process read`、`message`、`watch` 和 `audit` 能力，但没有 `CONTENT_READ` 或写能力；Investigator 可以读取内容；Artifact 可以发布工件；Orchestrator 才拥有完整编排能力。

创建前，内核同时检查父进程的 `role grant`、受信映像 `manifest` 和 `workflow admission`。创建成功后，`control id` 标识当前控制关系，`controller id` 把 `child` 绑定到同 `scope` 的根。用户传入的 `PID`、`role` 字符串或 `capability` 数值不能替代这些字段。访问另一个对象时，内核比较当前主体的 `scope`、完整 `lifecycle` 与 `control` 关系，避免仅凭 `PID` 复用授权。

3.3 七页 Context 地址区

每个 Agent 在固定 ABI 地址映射七页 Context。前六页由内核分配和发布，Guest PTE 只读；它们保存身份 `header`、最新响应、Context records、路径索引与可信 `mirror`。第七页是用户 `cache`，Guest 可以直接读写，用于保存高频策略数据。内核从不使用第七页决定 `capability`、`scope`、`cause`、`span` 或授权，因此同时满足“可直接读写”和“可信历史不可伪造”。

Context 容量固定为 128 条。记录包含 `sequence`、`request/tool/status`、`tick`、`cause`、`span`、

表 3.1: Agent 身份字段及安全用途

字段	用途	防止的问题
agent_id	当前 boot 内的 Agent 标识	普通进程冒充 Agent
control_id	不可由 Guest 指定的控制身份	PID 复用、越权 controller
role/capability	角色策略和具体动作权限	“知道工具名”等于可以执行
workflow id+generation	跨模块不可变 lifecycle key	旧 slot/watch/handle 命中新 workflow
VFS scope	文件和 metadata 可见域	跨 workflow 文件读取或写入
exec/storage account	workflow 资源归属	子进程或线程逃逸资源统计

branch、control id、短 payload/result 和 hash。内核用 publish sequence 与只读映射避免 torn read；用户还可以通过 query/snapshot UAPI 取得同一结构化历史。

3.4 Workflow lifecycle 与三类 gate

workflow identity 是不可变 id+generation。每个 lifecycle 保存 controller、member count、closing、active/departing operation、fence gate、fence sequence 和 Context branch allocator。slot 可以复用，generation 不能复用；所有 watch、资源账户、VFS sidecar、Task handle 和 artifact 都必须携带 generation。

我们用三类 gate 约束并发：operation gate 包围普通操作；departure gate 包围 exit 和 teardown；fence gate 形成精确 cut。fence 不在持锁状态睡眠：若已有 operation 或 departure 未清空，立即返回 RETRY。只有 closing&&members==0&&active==0&&departing==0&&!fence_gate 时才回收 scope、Context、watch、Evidence Ring 和账户。

更小的生命周期状态机

我们没有保留复杂、难验证的多阶段 retirement 产品语义。当前状态只公开 member、closing 和 gate；“retiring”只是内部满足回收条件的判断，不被包装成额外功能。

3.5 资源账户随 workflow 一起创建

workflow admission 分配 exec/storage resource domain 与 account；后续通过 agent_create_role 创建的 Agent 加入并共享该账户。process、thread、file object、filesystem block/inode、buffer、Agent state page 和 physical page 都按 workflow charge，ordinary 与 reserved class 分开。身份离开 workflow 时，资源不能因 fork 或线程增加而转为无主对象；controller close 和 context switch 也会触发空闲 credit trim。

这个设计为后续 Tool Phase Lease 和 workflow EEVDF 提供统一主体。如果只在工具层记录“某 Agent 调用了什么”，进程创建和页分配仍可能绕过资源边界；把账户绑定到 lifecycle 后，工具、VFS 和调度都能引用同一 key。

3.6 实现落点

- `os/agent_identity.c` 定义 role policy、身份发布、capability 与控制关系；
- `os/agent_core.c` 实现 Agent/workflow 创建、agent_info 和工具入口；
- `os/workflow_lifecycle.c` 实现 generation、member、close 和 gate；
- `os/agent_context.c` 分配、映射和维护七页 Context；
- `os/vfs_security.c` 把 process credential 与 workflow VFS scope 对齐；
- `os/resource_controller.c` 与 `os/workflow_credit_domain.c` 绑定资源账户。

3.7 验证

`agenttrust_ucore` 验证普通 `exec` 不能取得 Agent 身份、角色必须受 manifest 和父授权约束、权限只衰减不放大；`agentscope_ucore` 覆盖 scope 和生命周期 generation；`agentfinal_ucore` 覆盖 Context 固定映射、直接读取、用户 cache 写入、snapshot、rollback 和淘汰。Host mutation tests 进一步修改 manifest、UAPI 尺寸或生产对象清单，确认 checker 会失败，而不是只匹配成功日志。

任务一结论

我们完成了内核识别的 Agent 创建、独立 Context 和普通/Agent 共存。它是单 Hart、当前 boot、容量有界的实现；我们不声称跨重启保存 lifecycle 或 Context。

任务二：结构化工具、执行合同与 Task Channel

4.1 约束

结构化工具调用必须同时解决发现、参数、错误、安全和演进。一个只接受 `arg0` / `arg1` 的固定 `switch` 可以执行动作，却不能让模型可靠理解参数；一个只在用户态校验 JSON Schema 的框架又无法阻止恶意 Guest 直接调用系统调用。我们因此把模型友好的 `rich description` 放在 Guest，把紧凑、版本化、可强制的 `typed ABI` 放在内核。

任务二只要求至少三项工具。当前内核目录包含 25 项工具，并提供 `name/id` 唯一映射、V1 兼容描述、V2 `typed descriptor` 和结构化错误。数量不是主要贡献；关键是每条执行路径具有清楚且不被混写的保证。

4.2 工具发现和模型呈现分层

`sys_tool_list` 返回定长 V2 descriptor，包括 `version`、`size`、`tool id`、`name`、`schema`、`flags` 和 `side-effect` 类别。内核启动时验证目录完整性、`id/name` 唯一性和 `schema` 规则。调用者可以用 `name` 或 `id`，但二者同时存在时必须指向同一工具；未知工具、未知参数、缺失参数、类型错误、`size/version` 错误和缓冲区问题都返回稳定状态码。

Nexus 不把 25 项目录原样塞给模型。Guest runtime 从真实目录建立 `role-filtered view`，补充 `when-to-use`、`when-not-to-use`、参数、结果、副作用和错误说明，再按查询类别返回。运行时内部的 `wait`、`Context` 和 LLM 原语隐藏。这个分层使内核 `schema` 保持紧凑，模型描述保持可读，同时不让 `rich prompt` 成为授权来源。

4.3 Exploratory typed V2

V2 请求携带显式 `version`、`request size`、`tool name/id`、`parameter count` 和 `sized typed parameter` 数组。当前 `scalar type` 只有 `uint64` 与 `string`，降低教学内核解析面。内核一次性复制参数，验证 `key/type/数量` 后，才压平到共享执行器的 `arg0/arg1/64-bytepayload`。压平是执行器内部兼容层，不改变 UAPI 的 `typed` 校验事实。

V2 适合自适应 Agent Loop：模型可以先查询文件，再根据候选数决定读取摘要或委派。每次调用仍检查 `role/capability`、`scope`、`schema`、资源和工具自身不变量，但调用图没有提前冻结。因此我们称它为 `exploratory RPC`，不把它写成 V3 相同的合同保证。

4.4 Compact batch

`agent_run()` 接受最多 64 个 compact agent_op，在一次 syscall 中按数组顺序同步执行。它用于固定操作序列和 Context 压力验证，入口调用次数少，但不是 typed KV batch、并行 batch 或 non-blocking runtime。任何一项结果仍写入对应 Context；返回数组保留逐项状态。

这种兼容路径也是性能解释的必要边界。one-shot Task 实验比较的是等价 16 次空 ECHO 语义下的三种 transport sequence，而不是声称三条路径拥有相同 wire format 或相同已知复制字节。

4.5 ENFORCE V3 冻结执行合同

高风险、可预先声明的工作流使用 V3。每个 lifecycle generation 最多冻结一份版本 1 合同，最多 24 个拓扑有序节点；predecessor 只能指向更小 node id。节点冻结 tool、schema digest、required capability、input/output artifact type、deadline、attempt、retry/cancel、provenance/effect manifest 和 exec/storage envelope。

V3 保留 V2 prefix，同时绑定 contract generation、node/attempt、source node、精确 predecessor Context sequence 和 32-byte fingerprint。ENFORCE 激活后，合同外调用和受保护 direct syscall 在效果前被拒绝。每个 accepted node+attempt 使用稳定完成槽；合法重试返回原 terminal，不重复副作用。每份合同最多 48 个 attempt 槽。

4.6 Tool Phase Credit Lease

合同节点还可锁定工具峰值资源。进入工具前，Phase Lease 从账户现有 U 中原子锁定 exec/storage envelope，状态按 ADMITTED→ACTIVE→DEACTIVATED→SETTLED 推进。分配器发布对象前取得 nonce claim；失败对象把额度退回 locked U，成功对象继续持有同一 U，最终析构只结算一次。

Lease 不增加 quota，也不允许预测式超卖。它解决的是多类对象在一次工具内短时共同达到峰值时，如何保证 envelope 在副作用前可用；未使用部分 terminal 时转为 F。active phase 不能任意阻塞，全局 claim 和每 lifecycle phase 数都受合同 24 节点上限约束。

4.7 Typed Task Channel core

Task Channel 借鉴 SQ/CQ 通信形状 [6]，但实现为 AgentOS 自有 ABI。每 Agent 按需映射一个 16-slot SQ 与一个 16-slot CQ，另有 request/resource 两个内核私有页，共四页。SQE/CQE 均为 128 字节；内核先复制完整 SQE，再检查 ring/slot generation、单调 request id、contract/node/attempt、schema、deadline、cancel link 和 typed handle，避免共享页 TOCTOU。

私有 head/tail 是权威状态。CQ full 产生 backpressure；水位异常设置 sticky resync，issuer 必须显式恢复。每个 accepted target 在 success/failure/cancel/timeout 中只发布一个 terminal

CQE; cancel 自己不产生第二个 CQE。completion 携带 Context sequence 与 Evidence ticket, 但 “one terminal CQE” 不等于远程副作用的分布式 exactly-once。

当前 provider 同步、只接受 null input 和 output NONE; 八槽 typed resource 表已实现 generation、owner、digest、producer 与 provenance 重验, 但 RESOURCE_IMPORT fail closed, 没有业务 payload/result backend。Nexus TASK 和模型 wire 都不走这条 channel。

4.8 验证

agenttoolabi_ucore 覆盖目录发现、name/id 冲突、未知工具、缺参、错参、边界 buffer 和 structured response。agentcontract_ucore 覆盖正常 DAG、deadline、retry/cancel、source Context 与不可信 provenance 的 deterministic pre-effect denial。agenttask_ucore 让 compact batch、scalar V3、SQ/CQ 分别执行 16 次空 ECHO, 要求同一 tool/status/Context-proof/evidence-proof/zero-result fingerprint, 同时验证 full、resync 和 retained-terminal cancel。

one-shot campaign 保留 96 条 16-op sequence。中位时延是 batch 561 us、scalar V3 2051 us、SQ/CQ 1620.5 us。它们是 sequence 级 wall-time 区间; 逐 operation 的 Context service-start interval 只有 10 ms tick 分辨率, 不能均摊成伪造的单请求微秒。

任务二结论

我们交付了 25 项可发现工具和三类结构化执行机制。V2 提供自适应探索, V3 提供冻结合同, Task Channel 提供有界 SQ/CQ core; 当前 Task provider 和资源 backend 的限制在接口、测试与文档中均 fail closed。

任务三：Context Path、Provenance 与可验证证据

5.1 约束

多轮 Agent 需要把工具结果反馈给下一轮，但“记住文本”不足以形成可信 Context。我们必须区分谁产生了结果、它来自文件还是内核、通过哪条 route 跨 Agent、当前 active branch 是什么，以及一次 rollback 是否只是改变后续视图而没有撤销既有外部效果。

此外，题面要求超过五轮并在超长时自动淘汰。固定容量必须同时避免 torn read、引用已淘汰 predecessor 和把用户 cache 当成授权事实。我们用进程级 Context Path 保存运行历史，用 workflow Evidence Ring 保存 canonical evidence，再由 fence 给出外部可复核 cut。

5.2 Context 记录与因果路径

每次工具调用在效果边界形成 Context record。sequence 单调递增；记录绑定 request、tool、status、service-start tick、cause sequence、span、branch generation、control id、payload/result 摘要和 record hash。path_parent_sequence 指向当前 active head，successor index 支持分支查询。

Context append 采用 prepare、填充、publish、commit 的顺序。只读 mirror header 同时发布 count、head、oldest、latest、visible head、dropped 和 rollback count。读者先取稳定 summary，再读 slot 并重验 sequence/hash，最后确认 summary 未变化，从而拒绝 torn snapshot。

容量为 128 条。写满后按 sequence 淘汰最旧记录；active path 不能引用已淘汰节点。agentfinal_ucore 在一次 64-op compact batch 后继续执行 snapshot、branch、rollback 和 window 淘汰，超过题面五轮门槛。

5.3 Rollback 的精确语义

rollback 只把 visible_head 移到仍保留的某个 Context sequence，并为后续调用分配新 branch generation。它不删除历史 record，也不倒转已经发生的文件写入、IPC、进程创建或外部 provider 请求。clear 同样重置当前 Context 状态，而不是系统级事务回滚。

这个限制非常重要。如果把 Context rollback 描述成副作用回滚，用户会错误相信一次已批准动作可以被撤销。真正需要效果幂等或补偿时，应由 V3 attempt cache、工具协议或业务任务显式实现。

5.4 六类固定 provenance 标签

我们不让内核解析自由文本或运行 prompt-injection 分类器，而是使用六个可组合标签：

表 5.1: Context provenance 词汇

标签	含义	典型来源
KERNEL_FACT	由内核结构化接口产生的事实	PID、resource、scheduler、capability
TRUSTED_USER_CONTROL	经过产品控制面定义的用户控制	新用户 turn、精确审批决定
AGENT_DERIVED	Agent 从已有输入推导	摘要、报告、计划
UNTRUSTED_FILE_DATA	文件或元数据输入	query/read/digest source
UNTRUSTED_TOOL_OUTPUT	工具执行返回	V2 result、模型工具结果
CROSS_AGENT_DATA	经过 IPC 或工件跨角色	TASK result、broker artifact

文件、工具、IPC mailbox 和 Task resource 都执行保守 OR 传播。重新哈希、摘要或跨 Agent materialize 不会删除旧标签。send_message/llm_request/llm_response 允许这些带来源的观测继续进入 Agent Loop，但输出保留原污点，route、capability、schema 和 effect 检查不放松。其他副作用工具不会因为数据能被发送而获得额外权限。

5.5 Manifest 与 pre-effect gate

每个内核工具的安全 manifest 声明 accepted input labels、output-added labels、required capability 以及 file/metadata/IPC/process/permission/artifact/watch side-effect mask。启用 ENFORCE V3 时，外部效果必须同时满足 lifecycle generation、frozen contract edge、schema、capability、manifest 和 provenance。任何一项不满足都在效果前返回 DENIED，并把 source Context、tool、reason、lifecycle 和 evidence ticket 写入 critical evidence。

V1/V2 仍检查 capability、scope 与 schema，但没有 frozen-edge/provenance envelope。我们在文档和产品中保持这一区分，避免把 Nexus V2 的灵活性包装成 V3 的高保证。

5.6 Fence-Sealed Evidence Ring

每个活跃 workflow 按需分配四个 Agent state page：三页 ordinary ring，共 48 个 256-byte slot；一页 critical ring，共 16 个 slot。生产者 reserve 单调 ticket，在 IRQ-on 区域填充后 commit，失败则 discard 并记录 gap。读者遇到更早 BUSY ticket 时隐藏后续记录，保持发布顺序。

普通成功 Context 只写一次 canonical evidence，避免同一事实同时写 Context、ledger 和多个 hash chain。拒绝或授权效果进入 critical ring，并保留一条兼容 ledger 投影；ring 失败

则 fail closed 到 legacy ledger。audit/timeline/provenance 因而仍是合并视图，不是已经删除的旧接口。

环满时，已提交 segment 先滚入内部 root 再复用槽。显式 workflow fence 使用 SHA-256 domain separation，绑定 previous public root、challenge、fence sequence、ticket range、event/gap count、metadata generation、credit epoch、exact U digest 和 ordered segment digest。只有 challenge-bound fence 才推进外部 root；内部 rollover seal 不冒充 receipt。

5.7 Workflow fence 的精确 cut

controller 通过 `agent_run(count=0, AGENT_RUN_F_FENCE)` 发起 fence。内核依次检查 orchestrator/control、关闭 gate、取得 metadata quiescence、排空 live-query deferred work、处理 VFS deferred reclaim、trim exec/storage credit 并要求所有 P 为零，随后密封 Evidence Ring。receipt 在内部状态提交后才 copyout；copyout 失败的同 id 重试返回缓存结果，不重新切割。

320-byte receipt 明确标记 partial coverage、credit exact、evidence sealed 和 metadata volatile。Evidence Ring 是内存结构；sealed 不表示落盘、fsync、掉电原子或跨重启恢复，也不覆盖全部文件内容、Host 行为和每条调度事件。

5.8 验证

`agentfinal_ucore` 验证 64 次顺序调用、可信 mirror、query/snapshot、branch、rollback、淘汰和 record hash。provenance fixture 在 ENFORCE V3 下把不可信来源导向计划外副作用，要求 pre-effect DENIED 和 critical evidence；mutation tests 删除标签传播或 effect mask 时必须失败。fence tests 覆盖 challenge、retry cache、conflict/stale、credit P、metadata generation 和 copyout cut。

任务三结论

Context 是有界、可查询、带因果和来源的内核历史；Evidence Ring 与 fence 提供当前 boot 的 challenge-bound cut。它不证明文本语义安全，也不是持久日志或远程证明系统。

任务四：Agent Live-Query FS 与文件查询

6.1 约束

Agent 经常围绕“当前哪些对象满足条件、哪些对象刚发生变化”工作。普通目录遍历可以回答一次性问题，却不能表达业务 metadata、选择性索引、谓词变化和增量丢失后的恢复。直接修改磁盘格式又会显著扩大教学内核改动，并把当前实验性语义固化为持久 ABI。

我们选择 boot-scoped、显式登记的 Agent metadata。它覆盖文件属性、内容摘要/哈希、结构化查询和 typed live watch，仍由原 uCore VFS 管理文件内容。每次重启由 Guest 重新登记，避免把内存索引冒充持久数据库。

6.2 显式 metadata 与真实 inode 绑定

具有 META_WRITE 的 Agent 调用 `agent_file_meta_set()`，把 project、workflow、run、stage、status、kind、摘要和 dependency 等字段绑定到真实 dev+inum+incarnation。更新由 `update_mask` 限制；删除使用显式 DELETE 语义。普通 create/rename 不自动进入 catalog，避免为所有 VFS 对象支付维护成本。

incarnation 解决 inode number 复用问题。查询或 watch 命中记录时，内核重验对象仍是同一 incarnation 和 active workflow scope；unlink、内容大小变化或对象失效只更新已经登记的 sidecar。用户态传入旧 inode number 不能继承 metadata。

6.3 Scan、选择性索引与结构化结果

catalog 有界保留当前 boot 记录，并为 status/stage/kind 维护选择性索引。查询结构包含 predicate、scope、计划选择和最大 hit 数，返回结构化对象、候选数、实际检查记录数、结果数、generation 和 cache 状态。调用者可强制 traversal 或请求 index；即使使用索引，内核仍遍历实际候选，不把跨查询结果缓存伪装成索引命中。

这个设计同时提供正确性和可测工作量。若只报告微秒，QEMU/Host 抖动可能掩盖路径差异；候选数和 records examined 能说明 index 是否真正减少检查。结果上限为 8，防止一个查询无限占用用户缓冲区。

6.4 Typed live watch

`agent_live_watch()` 安装完整 `typed agent_file_query`，而不是解析短字符串。每次 `metadata before/after transition` 计算三种变化：不匹配到匹配为 `ENTER`；前后都匹配且记录变化为 `UPDATE`；匹配到不匹配或对象删除为 `LEAVE`。

事件通过 `AGENT_EVENT_FILE_QUERY` 进入目标 Agent 的 `Context` 和有界队列。订阅绑定 `workflow generation`、`scope` 和目标 `control id`，`PID` 复用不能继承。用户态 Agent Loop 可在没有变化时睡眠，在事件到达后增量处理，而不需要不断扫描目录。

6.5 增量丢失与 resync

事件队列和 `pending` 表都有固定容量。若内核无法可靠投递全部增量，它不丢一条事件后继续假装视图完整，而是推进单调 `resync_generation` 并合并发送 `RESYNC_REQUIRED`。Agent 必须重新 `snapshot/query`，再以对应 `generation ACK_RESYNC`。旧 `ACK` 不能清除更新的缺口。

`workflow fence` 会在 `metadata transaction` 中排空该 `scope` 的 `unlink/content pending` 和待投递 `resync`。若缺口仍未确认，`fence` 返回 `RETRY`，不把不完整 `generation` 写入 `receipt`。这个设计把“增量可能丢”转为显式协议状态，而不是依赖日志猜测。

6.6 文件查询 one-shot 实验

规范活动覆盖 `catalog size 24、64、96` 与 `exact hit count 1、2、4、8` 的完整 3×4 网格。每格有 15 个 AB/BA inner pair，共 180 个配对，没有插值。独立推断单位仍是 `source boot`：每个 `hit-count` 只有一个 `boot`，同 `boot` 的 15 个 pair 是重复测量，不能写成 15 台独立机器。

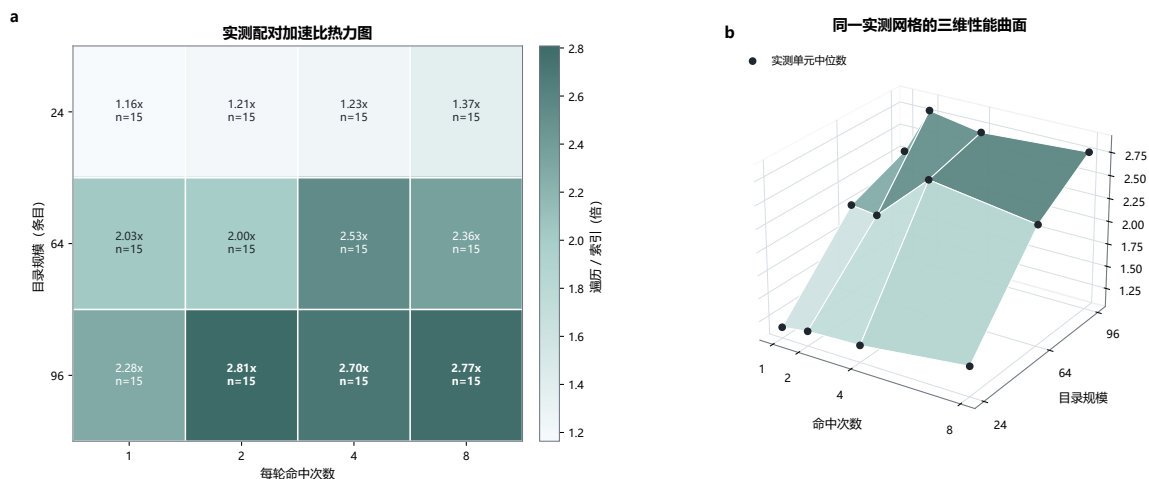
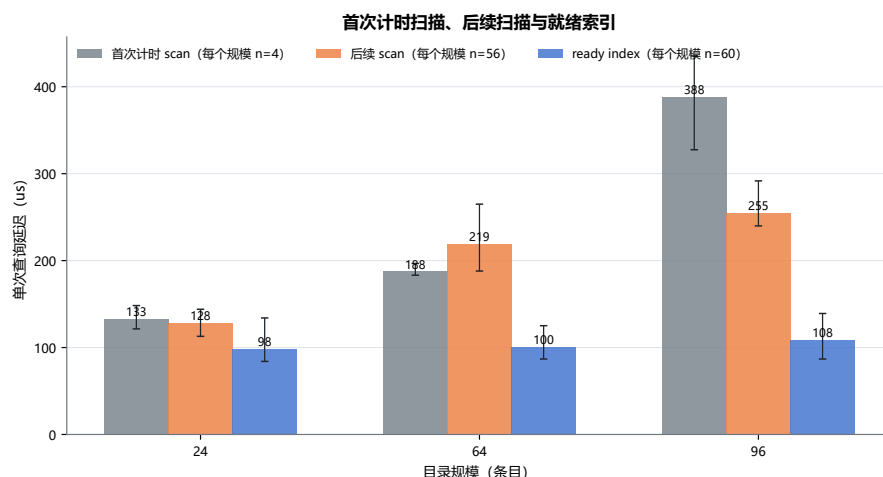


图 6.1: Catalog size 与 exact hit count 的实测 scan/index 加速比；每格 15 个 inner pair

12 个实测格的 median speedup 为 1.164–2.808 倍。该结果说明在当前 boot、当前 metadata corpus 和查询实现下，选择性索引持续减少 scan 路径成本；它不是跨硬件总体推断。三维 surface 与热力图使用同一网格，黑点才是实测位置，不能从曲面外推未测参数。



数据: agenteval_samples.csv; 柱为中位数, 误差线为 1IQR。首次显式 warmup 后的首个计时 scan 块; 数值按每轮命中次数归一到单次查询。每个 hit count 来自 1 次 fresh boot, 共 4 次; 图例 n=56/60 包含 boot 内重复测量。

图 6.2: 显式 warmup 后的 first/repeat scan 与 ready index 分组

全部 28 条 diagnostic 都是 cache=ready。图中的 first 是显式 warmup 后保留的第一个 timed scan, 不等于物理 cold cache; repeat 表示后续 scan, indexed 表示 ready index。我们保留这一命名, 避免把无法控制的宿主页缓存状态包装成冷启动实验。

6.7 实现与验证

metadata catalog、query planner/object lifecycle 和 typed event 分别位于 os/agent_metadata_catalog.c、os/agent_metadata_query.c、os/agent_metadata_objects.c 与 os/agent_live_query_events.c; VFS authorization 和 incarnation 位于 os/vfs_security.c。

agentfs_ucore 覆盖 set/update/delete、summary/digest、incarnation 和 scope; agentbench_ucore 以同数据集输出 scan/index 候选、records examined、result count 与 query tick; live-query checker 和 Guest 场景覆盖 ENTER/UPDATE/LEAVE、queue gap、RESYNC_REQUIRED、旧 ACK 和 fence drain。

任务四结论

我们实现了当前 boot 显式对象的结构化 metadata、选择性索引、内容摘要/哈希和 typed live query。它不是通用全文索引、持久数据库或自动索引整个 VFS。

任务五：Agent Loop substrate 与 Workflow EEVDF

7.1 约束

Agent Loop 的“循环”不应等同于 while 轮询。真正的运行时需要在等待用户、模型、文件变化、IPC 或 heartbeat 时让线程睡眠；事件到达后按可信 route 唤醒；多个 Agent 或多个 workflow 并发时，调度器还要避免线程放大、短睡眠投机和单一高频事件长期占用 CPU。

内核也不能越界成为 planner。我们提供 watch/wait、heartbeat、MESSAGE route、LLM correlation、lifecycle 和 workflow 调度 substrate，下一步工具仍由 Guest 策略或模型决定。

7.2 有界事件队列与 watch/wait

每个 Agent 具有容量 16 的事件队列，其中保留 kernel reserve，并限制同 source 最多 4 条排队事件。route 表同样有界。发送 MESSAGE 需要 MESSAGE_SEND capability、同 active workflow、显式 source/target route 与匹配 watch；内核把真实 source/target/control、corr 和 provenance 写入事件，不相信 payload 声明的 actor。

agent_wait 使用 thread-generation reservation，先检查已排队事件，再原子安装 waiter 并睡眠，避免“检查为空后事件到达、随后永久睡眠”的 lost wakeup。timeout、cancel 和 event consume 都更新 loop state 与 Context/audit。无事件时线程不忙轮询。

7.3 Heartbeat 是事件，不是内核业务

Agent 可以设置 heartbeat interval。timer 到期后，内核只产生 timer event 并唤醒 Agent；它不执行恢复脚本或模型逻辑。用户态收到 heartbeat 后决定检查状态、发送消息或继续等待。停止 heartbeat 会清理 generation-bound 状态，旧 timer 不能命中新进程。

这个机制覆盖长期会话的 liveness，但不提供分布式 failure detector 的准确性保证。heartbeat 延迟仍受可调度 safe point 和当前单 Hart 负载影响。

7.4 绑定身份的 LLM correlation RPC

LLM_REQUEST/LLM_RESPONSE 复用 MESSAGE 和 wait substrate。请求成功后，内核

登记完整 lifecycle、requester PID/control、relay PID/control、corr 和 deadline。corr 必须非零并对同 requester 身份严格递增；只有 MESSAGE 真正入队且 pending 发布成功才推进，失败投递不消耗编号。

只有原 Relay 在同 lifecycle 返回匹配 corr，才产生 LLM_DONE 并原子消费 pending。pending 使用 120 秒 kernel-tick TTL；全局容量为 NPROC，每 requester 上限为队列容量。有界 terminal history 在记录保留时区分 consumed replay 的 STALE 与 expired 的 TIMEOUT；覆盖后旧响应按 unmatched DENIED。该 RPC 不解析 prompt、JSON、HTTPS 或 MCP。

7.5 Workflow Credit Domain

资源账户维护 used(U)、pending(P)、free(F) 和 held=U+P+F。额度不足时按资源种类 quantum 从全局预充少量 F，热路径只执行：

```
reserve:F→P  commit:P→U  cancel:P→F  release:U→F
```

这些本地移动不改变 held，减少全局统计写入；补充 credit 前仍检查 global、class 和 account hard limit。压力路径、context switch、account close 和 fence 会 trim F。fence 在单锁中 trim+snapshot，要求全部 P 为零，导出精确 U 和 account slot/generation、credit epoch、U vector 的 digest。F 是预充额度，不是超卖。

7.6 按 workflow 聚合的 EEVDF

传统按线程公平会奖励创建更多线程的 workflow。我们把同一 resource domain 聚合为一个调度实体。调度器用 workflow vruntime 计算 lag，只有 vruntime≤global_vtime 的实体 eligible；latency class 和 event/deadline 缩短 service request，再从 eligible 集合选择最早 virtual deadline。dispatch 后按实际 service cycles 给 workflow 记账，睡眠 lag 向 global vtime 衰减。

一个实体走 fast path。身份不完整、内部不变量失败或容量外情况回退旧 RR/Agent heuristic，并计数 fallback；失败计划不修改 EEVDF 状态。实体表总容量为 4，其中固定包含一个 BOOT_SEALED bootstrap participant，因此最多同时有三个 fresh workflow。

所谓 1/4/16 场景必须精确解释：1 是单实体 fast path；4 是 bootstrap 加三个 fresh 的满容量；16 是四波逻辑样本，每波复用同一 bootstrap 并累计创建 12 个 fresh lifecycle，不是 16-way 并发，也不是每波四个 fresh。

7.7 调度实测

one-shot campaign 用六个 fresh QEMU boot 采集并发度 1、2、3、4 的 service window 与 exact wake probe。504 条 probe 直接读取 scheduler trace 的 ready_age，其中 425 条为 0 tick、79 条为 1 tick，没有右删失。tick 为 10 ms；这说明在该负载下全部 fresh waiter 在一个 tick 内被观察到调度，不等于微秒级 wake latency。

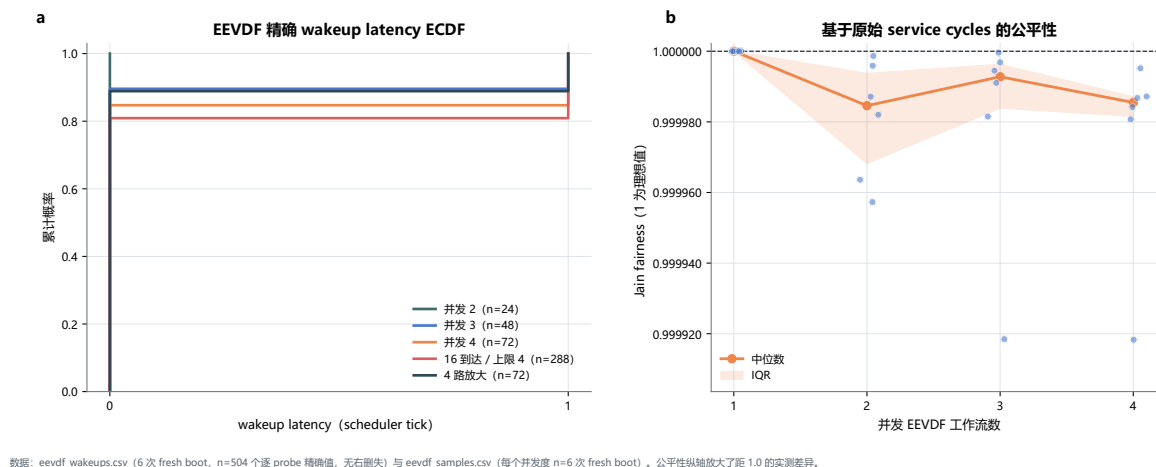


图 7.1: EEVDF exact wake latency 与并发 1-4 的 Jain fairness

以同一窗口 raw service cycles 重算的 Jain 中位数在并发 1/2/3/4 下分别为 1.000000、0.999985、0.999993、0.999985。并发 1 的 Jain=1 是平凡基线；图使用 focused y-axis 展开接近 1 的差异。公平数值只针对已采集 workflow service，不等于整机所有进程或跨硬件公平性。

EEVDF boot 05 有三条并发串口 sample summary 拼接。提取器保留 warning 并排除不完整 summary，504 条带完整前缀的 exact wake probe 全部保留；我们没有静默修补或插值。

7.8 实现与验证

os/agent_ipc.c 实现队列、watch/wait、route、heartbeat 和唤醒；os/agent_core.c 实现 LLM pending 与 Agent loop state；os/workflow_scheduler.c 实现 EEVDF；resource controller 与 credit domain 维护账户。agentloop_ucore 覆盖 atomic wait/wakeup、timeout、cancel 和 heartbeat；agentsched_ucore、agent_eevdf_ucore 覆盖 fast path、满容量、公平、线程放大、sleep decay、fallback 和 wake histogram。Host scheduler model 单独验证抽象 lag/deadline 算法，但不代替 Guest 固定槽位和真实 lifecycle 拓扑。

任务五结论

我们实现了事件驱动、无事件休眠、可信多 Agent IPC、模型 correlation 和 workflow 级调度。内核提供 Loop substrate，不解释目标或选择下一工具；hard deadline 在首个 schedulable safe point 结算，不承诺 wall-clock 终止上界。

任务六：确定性综合场景与可重复基准

8.1 约束

综合任务既要连接至少三个模块，又要在真实 QEMU 中完成，还要提供性能对比。若把模型随机性混入基准，重复运行的路径和工具数量可能不同；若只播放固定日志，又无法证明机制真的参与。我们因此把“可重复基准”和“自由交互产品”分成两条同等重要但职责不同的路径。

`contest-demo` 使用确定性 Guest policy，锁定业务工作量和 AB/BA 次序；Nexus 使用模型或 replay 动态委派，展示产品交互。前者回答“同负载下机制和路径如何表现”，后者回答“真实用户目标如何被内核治理”。

8.2 确定性科研恢复 workflow

`labdemo_ucore` 创建 Orchestrator、Sentinel、Investigator 和 Recovery，建立 workflow identity、MESSAGE route、Context、metadata 和 capability。场景围绕一个科研任务运行：查询项目与 run 状态、发现失败 stage、读取依赖和摘要、发送跨 Agent 消息、尝试受限动作、形成恢复选择并复核结果。场景登记 `prepare/align/analyze/report/archive` 的状态和依赖，完成 align 恢复与 report 更新；analyze 和 archive 保持 pending。

固定 policy 不是伪装成 LLM。代码显式安排阶段和参数，工具结果仍由内核真实执行并检查；任何失败都会使场景失败。它适合验证身份、Context、文件索引、IPC wait、权限拒绝和 recovery action 是否贯通，也适合在同一 AgentOS Guest 中对 traversal/compat 与 indexed/native 做业务合同一致的配对测量。

`labdemo_ucore` 输出的 `kind=fence quiescence` 只用于稳定性能计数。真实 320-byte workflow fence receipt 由双目标路径中的 `rp_agentos_orch` 生成，并由 `makedual-platform-run` 复验。

8.3 Native/Compat 与 AB/BA

综合场景保留 traversal/compat 与 indexed/native 两条功能等价路径。每个独立 QEMU boot 内按 A/B 或 B/A 顺序运行，记录共同 workload/result fingerprint，减少顺序偏差。核心测量区间覆盖查询、恢复写入、fsync 和复核；query block 单独报告纯查询区间。这两个窗口不能混为一个“查询延迟”。

makecontest-demo 自动构建 AgentOS Guest、启动 QEMU、解析串口并写出逐样本 raw、measurements.csv、summary.json 和报告。dual-platform-run 另构建 plain uCore 与 AgentOS-uCore，用确定性用户态合同做端到端比较。未实际运行时，命令存在本身不构成性能数字。

8.4 冻结 one-shot 主比较

规范活动在同一源码提交上执行 16 个独立 workflow QEMU boot，平衡 AB/BA。16/16 个 core interval 中 indexed 更快；traversal/indexed core median 为 34.713/13.294 ms，配对 speedup median 为 3.118 倍。

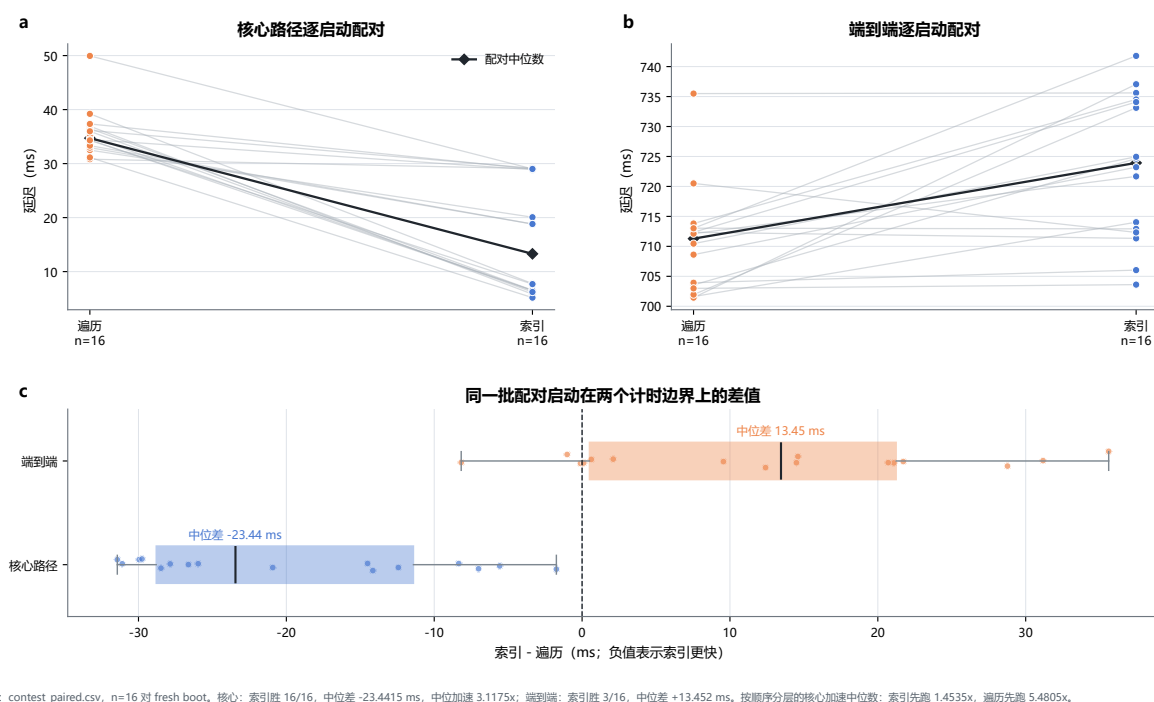


图 8.1: Core-path 与端到端窗口的配对结果及顺序分层

这组结果不能简化成“系统端到端快 3.118 倍”。端到端只有 3/16 个配对支持 indexed，indexed minus traversal 的端到端差值中位数为 +13.452 ms，即当前 end-to-end 窗口中 indexed 更慢。AB 与 BA 两个顺序分层的 core speedup median 分别为 1.4535 倍与 5.4805 倍，说明顺序效应显著。我们因此只声称 core path latency 在该负载中改善，并把端到端恢复、fsync、复核和测量噪声作为独立工程问题。

8.5 工作量与 I/O 口径

文件查询图中的 bytes_read 是 workflow actor core-window lane work；其他 contest numerator 是 shared global/end-to-end delta，再按 actor core-window observer syscall 归一。Task 路径则按 completed operation 归一。这些 owner、scope 和 window 不同的数字不能混成一个进程归因，也不能跨 metric 用热力图颜色比较绝对量。

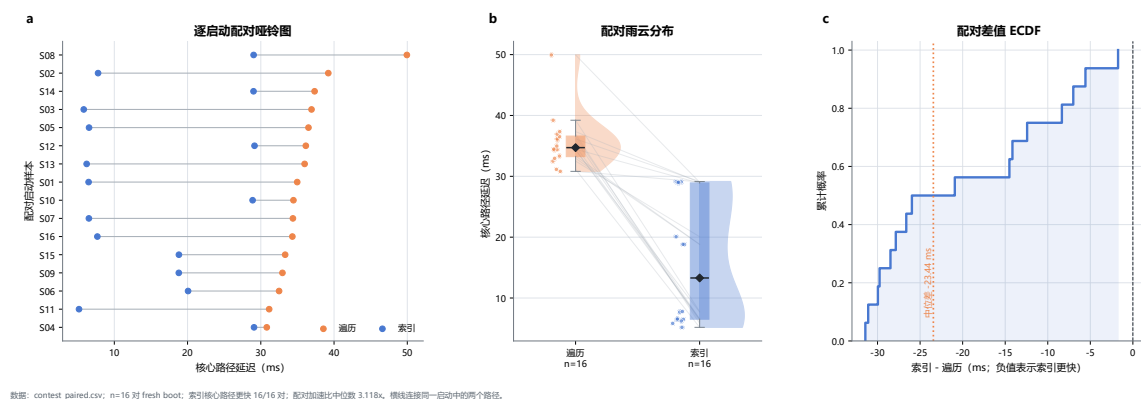


图 8.2: 16 个独立 QEMU boot 的 traversal/indexed core 配对原始分布

主文优先呈现配对 latency、catalog 网格、Task transport 与 EEVDF。混合 scope 的 I/O 热力图只放附录，并同时给 raw numerator、denominator、owner 和 window，避免漂亮的颜色掩盖测量主体差异。

8.6 与 Nexus 的职责分离

Nexus 不替代 contest-demo。模型可能因用户问题、历史、失败或审批改变任务图，这种动态性正是产品价值，却不适合作为等量性能基准。Nexus 可以读取历史 nexus_meas capsule 并在报告中引用已发布数字，但必须标记 historical_not_this_boot；它不会在启动时重跑 16 个 boot，也不能把 snapshot 改写成本次测量。

8.7 验证

Host runner 同时检查超时、panic、退出码、路径 fingerprint、AB/BA 样本完整性和输出 schema。one-shot manifest 记录源码 commit、Guest binary hash、工具版本、每个 boot 状态、33 个 raw 文件和 145-entry inventory；validation 要求 19 张长表 schema、数量、来源 SHA 和 10 组 PNG/PDF 全部一致。规范目录写入 COMPLETED 后采集入口 fail closed；离线重绘只读 CSV，不启动 QEMU。

任务六结论

我们交付了真实 QEMU 的多模块确定性综合程序和可复核性能数据，同时保留独立 Nexus 交互产品。当前最强性能结论是 indexed core path 在 16 个配对中全部更快；端到端结果并不支持同等级加速主张。

AgentOS Nexus：系统原生智能体协作中枢

9.1 从单次循环到长驻工作台

9.1.1 约束

单次演示可以证明模型会产生工具调用，却无法回答更重要的产品问题：多个具有独立身份的 Agent 如何持续协作，工具结果如何改变后续决策，操作系统如何对每一次等待、拒绝和资源消耗负责。如果 Host 代替 Guest 读文件、选工具或拼结结果，那么双窗口只是日志皮肤，不是系统原生协作。

9.1.2 设计

AgentOS Nexus 在一个 workflow/session boot 中长驻五个 Guest Agent：Coordinator、System、Research、Analyst 四个业务身份，以及一个只承载串口帧的 Relay。Coordinator 调用模型；三个专职 Agent 依据各自窄化策略执行真实 V2 工具。这种组合既保留“结果驱动下一步”，又避免把每个窄任务都伪装成独立 LLM 对话。

总体运行流程见图2.2：四个业务身份长驻，每次用户 turn 只动态创建子任务和 artifact。

9.1.3 实现

Host daemon 独占串口、TLS/API key 和 provider JSON 转换；左窗口 CLI 提交自然语言、展示工具请求并收集审批；右窗口 observer 只读取同一 Guest boot 的高信号投影。Relay 是唯一串口 frame writer，将 model、controller 与 telemetry 帧分流，不得选择业务工具。业务文件和 artifact 始终留在 Guest VFS 中。

9.1.4 验证

固定 replay 在一次真实 QEMU boot 中执行三个用户 turn，同时 attach controller 与 observer。严格 validator 检查四个业务 identity、逐请求 SHA-256、TASK 状态迁移、artifact 依赖、一次失败后重规划、一次审批拒绝及其零副作用，并要求 session 正常关闭后不再输出 telemetry。这证明的是 offline replay 的同路径闭环，不是云模型结果。

表 9.1: Nexus 四个业务角色的真实系统身份

产品角色	内核角色	职责	最小权限边界
Coordinator	Orchestrator	维护会话、调用模型、委派并验证返回	可编排，但所有效果仍经过审批与内核 gate
System	Sentinel	获取本 boot 进程、Context、文件大小和调度事实	无 CONTENT_READ；只执行固定无输入快照
Research	Investigator	验证本地资料与历史测量 capsule	读取指定 artifact，不执行网络搜索
Analyst	Artifact	消费 System/Research 结果并生成报告	只能发布自有 report；外部效果需审批

9.2 N1 通用子任务协议

9.2.1 约束

固定 pipe、阶段名或分号拼接文本无法支持动态委派，也无法将过期 handle、错误目标和跨 lifecycle 消息拒之门外。但是内核 MESSAGE 已经提供可控 route、scope、provenance、队列与唤醒，再在内核中理解研究任务又会越过机制与策略边界。

9.2.2 设计与实现

N1 是 Guest 用户态的版本化 fixed-size envelope: 44-byte canonical little-endian wire，以 N1：加无填充 base64url 通过 typed V2 send_message 传输。每条任务绑定 lifecycle id/generation、task id、parent task id、source/target、deadline、状态和有界值。状态机覆盖 TASK_ASSIGN、TASK_ACCEPT、TASK_PROGRESS、TASK_RESULT、TASK_FAILED 与 TASK_CANCEL。

内核只强制外层 MESSAGE 的可证属性；Guest runtime 负责 N1 canonical decode、transition validation、单项在途和 deadline。Host TASK_EVENT 是观测投影，不是 N1 wire；Task Channel SQ/CQ、MCP Task 对象和 V3 contract envelope 也不能与它互换。

短消息只携带控制和 generation-safe handle。模型产生的 objective、长工具结果和报告进入 artifact store。Research/Analyst 委派通过 TASK capsule 间接引用输入；System 的策略是固定 SYSTEM_SNAPSHOT，因此 ASSIGN 不含 input handle，也不为 Sentinel 增加 VFS 内容读权。

层次声明

N1 是 typed envelope over kernel MESSAGE。我们声称“内核治理了该通信”，因为发送者、接收者、route、scope、provenance、队列与唤醒由内核强制；我们不声称“内核理解 TASK 业务语义”。

9.3 Artifact 数据平面

9.3.1 约束

研究资料、测量数据和报告远大于内核短消息，而且数据流转不应把不可信内容升格为控制指令。仅有文件名也不够：重用 slot 或跨 lifecycle 读取都会产生混淆。

9.3.2 设计

Artifact manifest 绑定 lifecycle、handle generation、kind、task/parent、producer、owner、materializer、permission mask、payload size、source 和 provenance。Guest 以流式 SHA-256 覆盖完整 payload，读者重验 manifest 与 payload digest。System/Research 没有 artifact write，所以它们先返回可验证终态，再由 Coordinator 以“logical producer 与 materializer 分离”的 broker 语义写入结果。Analyst 只能发布自己的 report kind。

Research 复制历史测量时保留 UNTRUSTED_FILE_DATA；Analyst 合并两份输入时取 provenance superset，同时保留 UNTRUSTED_TOOL_OUTPUT 和 CROSS_AGENT_DATA。哈希只证明 Guest 读回的字节与当时 manifest 一致，不是内核签名、授权凭据或不可篡改 seal。

9.3.3 黄金场景的真实数据依赖

黄金场景限定为“结合当前 AgentOS 系统事实与本地历史研究资料，形成带来源的性能分析”。strict replay 中，System 工件 handle 为 65540，Research 工件 handle 为 65544。当次 System 投影包含 source=nexus_state、process_count=5、context_count=15、file_bytes=279 与 sched_budget=8。这些数字是该 replay boot 的事实，不是通用性能基线。

Research 读取的 schema v2 发布快照绑定 source commit：

```
2b14fb1f74b9bd093e6de939a16554620835699e
```

其规范字段为 records=96、traversal_us=34712.5、indexed_us=13293.5、paired_ratio_median=3.118 和 wins=16/16，并显式标记 historical_not_this_boot。Analyst 必须同时引用两个 handle 及其完整 digest。删除或更改 seed 时，读回校验、任务结果和报告均会随之失败或改变。

9.4 动态工具呈现与模型回路

9.4.1 工具目录不等于 prompt

内核的 25 项目录服务于 ABI 发现，不适合无差别地每轮发给模型。Nexus 首先调用 tool_list 并检查名称/ID/schema，再依据 product role、kernel role、capability 和副作用

策略筛选。tool_search 按类别分页返回 rich description：何时使用、何时不使用、参数与结果字段、常见错误和副作用。

默认只读面包含 pid_info、ctx_stat、query_process、get_system_status、read_context、query_file、read_file_summary、read_file_digest、dependency_query、capability_check 和 read_message。agent_wait、context_push、llm_request、llm_response 属于 runtime 内部原语，不对模型暴露。Nexus 的 delegate_task、read_artifact 和 publish_report 是 Guest 产品动作，其 pseudo ID 不会冒充内核目录 ID。

9.4.2 真正的观察后再决策

Coordinator 将目标、有界历史、已筛选 schema 和上一个工具结果发给 provider。在 WAITING_LLM 期间，内核挂起 Agent；模型帧到达后唤醒。当模型返回委派，Coordinator 创建 task，等待 ACCEPT/PROGRESS/terminal，校验 source PID、role、lifecycle、deadline 和 transition，再把稳定、已验证的投影回灌给模型。一次 stale handle 的 Research 委派会返回结构化失败，Coordinator 可创建不同 task 重新规划，而不是退出 workflow。

会话保留最近 4 个完整 turn，每个 turn 最多 8 个 model round；旧回合进入结构化 summary。为使 digest-bound replay 可复现，System 完整 artifact 保留动态 dispatch/budget-used/vruntime，模型历史只保留同一实际 artifact 中的稳定决策字段与 sched_budget。真实 payload size 保留在 controller tool event，而历史显式写作 value1_omitted=volatile_payload_size，不把 0 伪装成大小。

9.5 调用级审批、取消与安全降级

9.5.1 参数绑定审批

publish_report 不接受“本会话曾经批准过这个工具”这类粗粒度证据。审批请求绑定 session、turn、request、correlation、positive tool ID、canonical arguments digest、一次性 nonce、issued tick 和 expiry tick。Guest 收到决定后逐字段比对，并在副作用之前再检查有效期。只有审批通过后，Coordinator 才用 typed V2 artifact_update 更新已登记的精确 selector。

黄金 replay 特意拒绝发布：模型收到 DENIED，报告 artifact 仍可读，但内核 artifact_update 执行次数为 0，会话继续并返回安全降级的最终答案。

9.5.2 取消和关闭

Ctrl-C 取消当前 turn，不杀死 QEMU 或其他长驻 Agent；/quit 才执行 session close。取消是协作式的，已完成的工具副作用不会自动回滚。正常关闭时，Coordinator 停止 observer producer、最终 drain audit、join observer、关闭 telemetry writer 产生 EOF；Relay pump 读到 EOF 并 join 后才输出 SESSION_CLOSED。这保证关闭标记之后没有延迟 frame。

9.6 独立内核观测面

Observer 从 `agent_timeline_wait/read`、`agent_audit_query`、`agent_info` 与 Context snapshot 投影两类事实。`kernel_audit` 只转发真实 MESSAGE enqueue/consume 记录，保留全局 audit sequence、生命周期、PID/agent/control identity、source/target、correlation 和内核 status。MESSAGE audit 的 flags 本来为 0，投影也保持 provenance=0，不伪造非零标签。

`kernel_snapshot` 是每个业务 Agent 的 self `agent_info`，包含独立 Context sequence、wait sleep/wakeup delta、scheduler dispatch/budget/used/vruntime、capability mask 和可验 control identity。Audit sequence 不会被冒充 Context sequence。Coordinator 在 assignment 后及每次消费 worker reply 后同步 drain 共享 audit cursor，observer thread 也通过独立 mutex 使用同一 drain，以避免有界 principal history 在高频 MESSAGE 间丢失成对记录。

产品解释

左窗口说明“Agent 想做什么”；右窗口说明“操作系统实际允许、拒绝、调度和记录了什么”。observer 是有扰动的高信号投影，不是完整无损 trace，也不用于产生第十一章的性能数字。

9.7 交互控制面

第一版不追求复杂 TUI，而是使用稳定的终端产品语义。普通文本作为新用户 turn；`/tools`、`/agents`、`/tasks`、`/artifacts`、`/context`、`/status`、`/reset` 和 `/quit` 走 Guest control path。Host 不用缓存业务结果伪造这些命令。`/reset` 清理对话历史，但不新建 lifecycle，也不回收已发布 artifact slot。

- `makeagentos-nexus-check`：只运行静态/Host 合同。
- `makeagentos-nexus-replay`：运行固定 fixture 的真实 QEMU 闭环。
- `makeagentos-nexus-deepseek`：显式人工 live 入口，只以当次 API 往返与 Guest 日志作为成功证据。

同路径 replay 已可复验；provider 失败不静默切换另一模式。

跨模块安全与调度治理

10.1 安全不可约化为一张 allowlist

10.1.1 约束

Agent 安全的失败往往不是“工具名称不在白名单”，而是身份可伪造、跨 workflow 对象被重用、不可信工具文本进入控制流、审批被重放，或者在资源预留后只回收一部分状态。因此我们不把某一个接口宣传为“安全开关”，而是建立彼此独立的 gate 链。

10.1.2 授权链

一次工具调用至少经过以下约束：

1. 可信 exec/Agent identity 确定当前 PID、agent id、role 与 control identity；
2. lifecycle 必须活跃，且调用者不能跨过 generation-safe workflow scope；
3. typed schema 必须正确，参数数量、键、类型和长度在副作用之前校验；
4. role/capability 必须允许该 action，精确路由及 VFS scope 必须允许该对象；
5. provenance 不能越过节点允许的边界，现有不可信标签只能保留或增加；
6. 有副作用的具体调用还必须拥有未过期、参数绑定的审批；
7. 资源账户必须能完整预留，失败时不得留下部分提交。

这些 gate 不相互替代。人工批准 artifact_update 不会绕过内核 META_WRITE；持有 capability 也不会把 UNTRUSTED_FILE_DATA 改成 TRUSTED_USER_CONTROL。

图2.3用相同工作流的 Plain/AgentOS-uCore 对照展示了这些 gate 为何必须是内核可强制的系统属性，而不是 prompt 中的约定。

10.2 Provenance 作为流转属性

10.2.1 设计

AgentOS 将 provenance 放在 Context、IPC、工具请求、artifact 和 Evidence 之间传递。核心标签区分内核事实、可信用户控制、不可信工具输出、不可信文件数据、跨 Agent 数

据和派生数据。判定原则不是“数据由另一个 Agent 生成，所以可信”，而是保留来源的不确定性并增加 CROSS_AGENT_DATA。

V3 contract 把节点许可 provenance mask 冻结在合同中；V2 保留探索式循环，但每次调用仍受内核当前身份与标签检查。Nexus 的 Research/Analyst 在 Guest artifact 层进一步验证 superset，是产品层的 defense in depth，不把这一部分误写成新的内核安全模块。

10.2.2 威胁与对策

表 10.1: 当前系统中主要威胁与 fail-closed 对策

威胁	攻击路径	强制与结果
身份伪造	普通进程自报 Orchestrator 或 worker role	可信 exec policy、PCB identity 与 一次 control binding；PID 变化时 继承的用户态 active identity 失效
跨 scope 读写	持有旧 PID/handle 后 访问新 lifecycle	lifecycle generation、scope gate、 VFS owner 与 handle generation 共 同拒绝
Prompt injection	工具或文件文本包含 伪控制指令	UNTRUSTED_TOOL_OUTPUT/UN TRUSTED_FILE_DATA 保留；控 制与数据分离
审批重放	将旧的“允许此工具” 用于新参数	session、turn、request、correlation、 tool、digest、nonce 和 tick 全绑定， 过期即拒绝
部分资源提交	一个 vector 中前项已 扣账，后项失败	全量 preflight，再在单锁临界区移 动 U/P/F；失败无部分副作用
结果伪造	Host 自行读文件或生 成 tool_result	Guest 自主 V2 执行，model request SHA 绑定，artifact 完整 digest 重 验
观测伪造	Host 将自产事件冒充 kernel audit	kernel source 需 Guest 转发真实 record sequence/identity；Host schema 拒绝越界字段
关闭后延迟输出	telemetry thread 在 session closed 后续写	producer final drain/join、pipe EOF、 Relay pump join，最后才发 close frame

10.3 U/P/F 资源账户

10.3.1 约束

仅在调用结束后记账无法阻止超配；仅维护一个“已使用”数字又无法区分运行中占用、未来提交与可回收缓存。多资源一起预留时，任何一项失败都不能留下其余项的半完

成账务。

10.3.2 设计

每个 lifecycle 的资源账户分为：

$$H = U + P + F, \quad (10.1)$$

其中 U 是已提交使用量， P 是已预留但尚未提交的量， F 是可回收缓存。hard limit 检查 H 而不是只检查 U 。reserve 从可用配额进入 P ，commit 把 P 转到 U ，cancel/release 不会造成重复返还。系统压力下只能 trim F ，不能偷偷清理已承诺的 P 。

资源 vector 先验证所有维度，然后在锁中一次移动。workflow fence 要求 $P = 0$ ，同时对 storage/exec cache 执行单锁 trim，导出 exact U 。因此 receipt 中的资源根与截面对应，而不是事后估计。

10.4 Phase Lease 与安全包络

V3 frozen DAG 适合对预知过程给出高保证，但不适合规定一切探索式 Agent Loop。Phase Lease 在二者之间提供有界包络：租约绑定节点或阶段、工具范围、副作用 mask、provenance 上界、deadline 和资源预留。租约过期或阶段切换后，旧授权不得继续使用。

我们因此将 V3 称为“高保证执行模式”，将 V2 称为“受内核逐次治理的适应执行模式”。前者不是一般 Agent Loop 的唯一实现，后者也不等于无约束执行。

10.5 Workflow EEVDF 与防线程放大

10.5.1 约束

如果只在线程级调度，一个 Agent 可以通过创建更多 worker 获得更多 CPU 份额，这与“按 workflow 治理”矛盾。LLM、IPC 和 heartbeat 又会产生频繁睡眠/唤醒，没有对 lag 的处理容易出现唤醒突发。

10.5.2 设计与实现

AgentOS 把同一 workflow 的 runnable 线程折叠为一个调度实体，使用 lag eligibility 和 virtual deadline 选择；workflow 内再按已有进程策略选择成员。新建线程不会产生新的跨 workflow 份额。睡眠期间的历史 lag 按有界规则衰减，唤醒时将 ready age 写入调度 trace。

第七章的 6 个独立 boot 展示了 504 个 exact wake probe：425 个是 0 tick，79 个是 1 tick，无插值。并发 1/2/3/4 的 Jain 中位数分别为 1.000000、0.999985、0.999993 和 0.999985。这证明测量场景中的工作量分配，不证明任意硬件或任意数量 workflow 的统计性能。

10.6 Evidence Ring 与 Workflow Fence

Evidence Ring 使用固定页面和固定槽，ordinary 与 critical 分区，通过 reserve/fill/commit/discard 避免观察到半填充记录。critical deny/authority 不与普通成功共用一个淘汰窗口。当生命周期需要发布截面时，workflow fence 按 metadata/live-query drain、VFS cut、credit exact、Evidence seal 的顺序执行，并返回带 request/challenge 的 receipt。

该 receipt 使用 flags 如实披露 partial coverage 和 metadata volatile，因此“已 seal”不等于整个世界已持久。保留的内部 retirement 也不能冒充显式 workflow fence。这一设计使评委能看到“系统在什么边界上给出证据”，而不是仅看到一串成功日志。

10.7 安全与可用性的取舍

严格 gate 会带来更多结构化失败，这是设计结果而不是需要掩盖的异常。我们将 BAD_PARAM、DENIED、STALE、TIMEOUT、NO_SPACE 和 IO_ERROR 送回对应 Agent，使策略可以更改参数、改用只读路径或终止当前任务。不可恢复的身份或 lifecycle 冲突则 fail closed，不会为演示继续伪装成功。

对高保证固定流程，我们选 V3 contract；对需要观察后改变计划的 Nexus，我们选逐次内核治理的 V2。这一取舍保留了产品的可用性，同时没有把“模型可动态决策”误写成“模型可以绕过系统”。

功能、可靠性与性能验证

11.1 证据分层

我们不用单一“tests passed”概括整个系统。不同层的证据回答不同问题，下层通过不会自动升格为上层结论。

表 11.1: AgentOS-uCore 的六层验证证据

证据层	能够回答	不能回答
Focused checker	关键调用顺序、UAPI 布局、生产对象边界是否漂移	真实 RISC-V syscall 语义
Host/mutation 模型	账户、ring、resync、relay schema 与错误输入是否 fail closed	Guest 等待、调度和 VFS 实际行为
Cross build/link	当前生产对象能否在 RV64 编译链上链接	程序运行时会走到正确路径
Stack/module gate	调用图的栈预算和生产模块依赖是否越界	全部时序竞态已被排除
QEMU Guest	identity、Context、V2/V3、IPC、VFS、wait/wake 在实际 Guest 中是否闭环	其他硬件或任意工作负载的泛化
配对性能	指定版本、工具链、QEMU 和 Host 上的配对工程表现	某个内核机制的单独因果效应

11.2 功能验证矩阵

表 11.2: 六项任务的关键功能与证据入口

任务	核心约束	实现证据	动态验收焦点
一	内核可识别 Agent，不信任用户自报	PCB identity、trusted exec、Context mirror、lifecycle 与资源账户	create、fork、exec、exit、generation reuse、关闭门禁

任务	核心约束	实现证据	动态验收焦点
二	结构化参数先验证后副作用	25 工具目录、typed V2、compact batch、V3 contract、Task SQ/CQ	错误 schema/capability/provenance/deadline 拒绝；96 个 sequence 性能表
三	Context 和 Evidence 不得被用户直改	7-page Context、rollback、provenance、Evidence Ring、workflow fence	push、query、rollback、critical 保留、seal 幂等
四	查询走真实 VFS 和当前 boot 索引	显式 metadata、scan/index 一致性、typed watch、resync generation	create、unlink、incarnation、scan/index 结果相同、3×4×15 负载
五	等待不忙轮询，线程数不放大 workflow 份额	event queue、LLM correlation、heartbeat、workflow EEVDF	timeout/cancel、ready-age、504 exact wake、Jain fairness
六	同一 AgentOS Guest 的 traversal/indexed 工作量对齐，产品闭环与性能基准分工	deterministic labdemo_ucore、Nexus、Host validators、AB/BA runner	16 paired boot；Nexus 3 turn/4 identity/failure/replan/deny

11.3 可靠性与负路径

11.3.1 静态合同与 mutation

Focused checker 不只搜索“函数名存在”，还约束生产调用链和禁止的旁路。例如，workflow fence checker 锁定唯一 syscall cut 和 metadata/live-query、VFS、credit、evidence 的顺序；Nexus checker 锁定 System no-input 路径不读 TASK capsule、Research/Analyst 验证 handle 和 provenance、副作用审批的全字段绑定。mutation tests 改动长度、版本、顺序、digest、lifecycle 或身份字段，检查系统是否拒绝。

11.3.2 构建与栈安全

agent-uapi-check 对齐内核和用户头文件的 size/offset；agent-module-check 检查真实生产 object list 和依赖；kernel-stack-check 与 user-stack-check 在实际编译调用图上检查栈深。Nexus Guest 使用大型 static buffer 和专用小结构 history，避免将

24 个 TASK event 嵌入每个 turn 而突破 uCore 当前 MAXFILE=274432 bytes。Image target 在 mkfs 前以 raw binary 大小 fail closed，不通过扩大 FS ABI 规避问题。

11.3.3 QEMU 负路径

下列结果是产品语义的一部分，不是可以从报告中删除的“异常噪声”：

- malformed typed KV 在工具运行前返回 BAD_PARAM；
- capability、route、scope 或 provenance 不符合时返回 DENIED；
- 旧 generation handle 或已消费 response 返回 STALE；
- event/LLM/task deadline 超时返回 TIMEOUT，而不无限忙轮询；
- artifact 容量或 MESSAGE source queue 受限时返回 NO_SPACE；
- 审批 deny 作为结构化 tool result 回灌，不终止整个 Nexus session；
- observer 正常关闭前经过 final drain/join/EOF，关闭之后的任何 frame 都使 strict validator 失败。

11.4 Nexus 严格 replay 验收

11.4.1 方法

agentos-nexus-replay 不是 Host 重放一段录屏，而是启动当前 RV64 镜像、连接 controller/observer，将每条 fixture response 与 Guest 当次产生的 canonical MODEL_REQUEST SHA-256 绑定。请求文本、history 或稳定 tool projection 发生任何变化，原 fixture 都会 fail closed。

Validator 同时检查 ready-before-data 与 active request 不复用。因果顺序必须为 request < response < effects < next request。每个 TASK 只有一个 terminal，parent task 必须在当次 transcript 中存在；controller 与 observer 的 TASK 安全 metadata 子序列一致，kernel audit enqueue/consume 成对，Context sequence 不与 audit sequence 混淆。

11.4.2 黄金路径

三个连续用户 turn 覆盖：

1. System 执行无输入本 boot 快照，生成真实 System artifact；
2. Research 先因 stale/missing handle 结构化失败，Coordinator 创建新 task 重新规划，后续从本地 measurement capsule 生成 Research artifact；
3. Analyst 读取两份已验 artifact，生成带 handle/digest/source 的 report；发布请求被人工拒绝，但最终答案仍安全完成。

该路径要求 worker 在等待时真实进入 WAITING_EVENT，Coordinator 等待 provider 时进入 WAITING_LLM。成功路径不允许 Host 选择工具、读 Guest 业务文件或伪造 artifact。严格 replay 能复验此闭环；它不等于 DeepSeek live provider 已在该次测试中成功。

11.5 一次性性能数据集

11.5.1 数据来源与独立单位

规范运行位于 one_shot_metrics/data/20260811，由 CANONICAL_RUN 指向。采集共使用 30 个 fresh QEMU boot，保留 33 个 raw 文件，生成 19 张长表和 7,498 行结构化记录。每行保留 raw 相对路径、来源 SHA-256 和来源内序号，因而图表可回到原始串口证据。

表 11.3: 一次性采集的实验族与正确独立单位

数据族	设计	独立单位	关键限定
traversal/indexed	16 个平衡 AB/BA paired boot	boot	core 与 E2E 分开报告
file catalog grid	$3 \times 4 \times 15 = 180$ inner pair	boot 为推断单位	同一 hit-count 只有 1 个 source boot
compact batch/scalar V3/SQ-CQ workflow	4 boot $\times$ 8 rotating round	boot/block	每序列 16 op，共 96 sequence
EEVDF	6 boot，并发 1–4	boot/window 或 wake probe	504 exact probe，不从桶插值

同一 boot 中的 inner pair 是重复测量，不是独立机器实验。本次数据定位为工程探索与文档绘图，不进行跨硬件总体推断。

11.6 Core 与端到端窗口

16 个独立 paired boot 中，indexed core 在 16/16 配对中更快；traversal/indexed core 中位数分别为 34.713 ms 和 13.294 ms，配对中位加速比为 3.118 $\times$ 。但端到端窗口仅 3/16 个配对 indexed 更快，indexed-traversal 的中位数为 +13.452 ms。因此我们只宣称 core path 在该负载下获益，不宣称整个系统窗口已加速。

AB/BA 顺序分层的 core speedup 中位数为 1.4535 $\times$ 和 5.4805 $\times$ 。差异提示顺序、缓存或其他 boot 层因素仍然很强，正是我们保留平衡配对和原始分层的原因。

11.7 File query catalog 工作量

目录规模 24/64/96 与 exact hit count 1/2/4/8 形成 12 格，每格 15 个 AB/BA inner pair。图6.1与图6.2展示索引对不同选择性的影响。图中 first scan 是显式 warmup 后的第一次 timed

scan，不被伪称为可证明的物理冷缓存。同一 hit-count 只有一个 source boot，所以热力图是工程地形，不提供 15 个独立 boot 的置信结论。

11.8 Task 通道序列分布

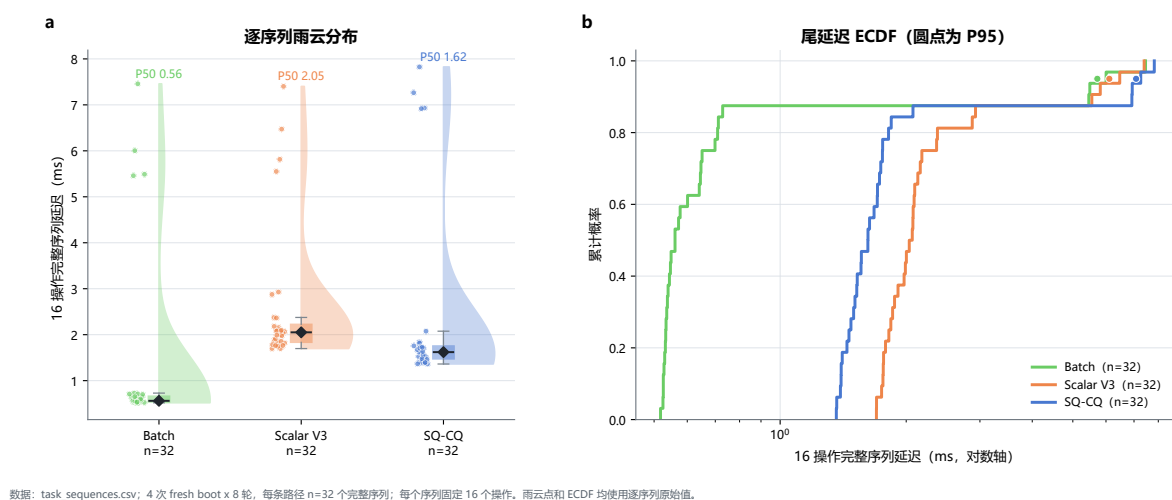


图 11.1: 4 个 boot、8 个旋转顺序下 96 个 16-op sequence 延迟分布

Compact batch、scalar V3 与 SQ/CQ 的 16-op sequence 中位数分别为 561 us、2051 us 和 1620.5 us。这是完整 sequence 的 Host/Guest 测量，不应将 10 ms tick 粒度的 service_start_interval_tick 均摊成伪造的单请求微秒延迟。此次数据显示了不同路径的产品代价，而不是声称异步通道必然快于同步 syscall。

11.9 EEVDF 唤醒与公平性

图7.1同时展示 exact wake ECDF 与 Jain fairness。在 504 个带完整前缀的 probe 中，425 个 ready age 为 0 tick，79 个为 1 tick。EEVDF boot 05 有 3 条并发串口 sample summary 拼接；提取器保留 warning 并排除不完整 summary，504 条 exact probe 全部保留，没有插值或静默修补。

并发 1 的 Jain=1 是平凡基线；scenario 16 的四波不是 16-way physical concurrency。我们在报告中保留这些限制，防止图形的美观性取代实验定义。

11.10 结果解释的最小规则

引用性能数字时必须同时披露

对外引用必须给出 target、source commit、工具链、QEMU/Host 环境、样本数、单位、聚合方式和失败/warning。paired boot 才是 core/E2E 比较的独立单位；inner pair 不能冒充 fresh boot。Nexus 的 nexus_meas 只是历史快照，它不能替代新的 contest-demo 采集。

已知限制与适用范围

12.1 产品能力边界

我们将 AgentOS-uCore 定位为“系统原生 Agent 机制与可交互垂直闭环”，而不是通用生产级多租户 Agent 云。本章列出当前的硬边界，便于评审者区分“已实现机制”、“有界产品场景”和“后续可拓展方向”。

12.1.1 Nexus 不是任意多 Agent 平台

Nexus 固定为 Coordinator、System、Research、Analyst 四个业务身份和一个 Relay。只有 Coordinator 调用模型；三个专职 worker 是拥有独立 PID、Agent/control identity、Context 和 role/capability 的确定性 policy Agent，并与 Coordinator 共享 workflow lifecycle、scope、资源域和调度实体。它们不是三个独立 LLM session。动态性体现在 Coordinator 依据目标与真实结果决定是否委派、委派给谁以及如何重规划，不体现为任意拓扑的自动扩缩容。

System 仅执行固定无输入系统快照，不拥有 CONTENT_READ；Research 检索本地已登记资料，不使用 Web Search 或外部论文 API；Analyst 不执行任意 shell。这些限制是最小权限设计，不是缺少的“隐藏开关”。

12.2 N1 协议边界

N1 是 Guest 用户态 fixed 44-byte canonical envelope，内核不解析 TASK kind 或 objective。它通过已有 MESSAGE 管道获得身份、route、scope、provenance、队列和唤醒保护，但当前不提供分布式 exactly-once、持久任务恢复或跨 boot 重放。TASK deadline 存入 32-bit tick，未承诺穿越低 32 位回绕长期运行。

每个 specialist task 将 worker reply 压缩在有界 MESSAGE 数之内，Coordinator 在 assignment 和每次 reply 后 drain audit。这是对当前内核 principal history 容量的明确适配，不是高吞吐分布式消息队列。持续 NO_SPACE 会有界 yield/retry，最终由 task deadline 使 Coordinator 超时收口；不保证拥塞下保留每一条中间 progress。

12.3 Artifact store 边界

- store 是 workflow-scoped、boot-volatile Guest VFS 存储，不是跨启动持久数据库。
- SHA-256 用于完整 payload 一致性重验，不是内核签名、来源证书、per-object ACL 或

immutable seal。

- 当前每 lifecycle 只有 32 个 generation-safe slot。三个 seed 占用前三个；System 每次委派消耗 1 个 result slot，Research/Analyst 每次消耗 capsule+result 两个 slot。
- 失败、超时和取消不回收已分配 slot。三个 seed 之后，全 System 最多 29 次，全双 slot worker 最多 14 次并剩 1 slot；混合场景按实际工件消耗。
- /reset 不新建 lifecycle，也不回收 slot。容量用尽时返回 NO_SPACE，需正常结束并启动新 boot。
- handle 只保留 lifecycle generation 的低 16 位，当前产品 boot 使用 generation 1，不声称无限期 daemon 安全性。

12.4 会话与模型边界

单个用户 turn 最多 8 个 model round，会话保留最近 4 个完整 turn，较旧内容进入有界 summary。本版不实现任意长上下文、泛用 token 压缩策略或完整 provider reasoning-content 回放。工具参数必须完整接收并通过 schema 后才执行；文本 streaming 不是当前验收前提。

Replay provider 使用请求 SHA-256 与固定 response 验证同一 Guest/内核路径。它不访问网络，不会被报告为云模型。DeepSeek 入口依赖当时网络、API key、供应商服务和模型行为；provider 失败不静默切 replay。本文只将已固定 fixture 的结果表述为 strict offline replay，不虚构一次 live provider 成功。

12.5 观测边界

Nexus observer 是同一 Guest/QEMU boot 中的只读 high-signal view，不是独立安全域、全量无损 trace 或性能采样器。它本身会调用 timeline/audit/agent-info，产生调度和串口扰动。因此第十一章的性能数字来自独立 one-shot 采集，不来自 observer 窗口。

Observer 只投影业务相关 MESSAGE audit 和每个角色的 self snapshot；未展示不意味未发生。Audit sequence 是全局 ledger 序号，Context sequence 是某个 Agent 的 Context Path 序号，两者不可互换。当前 observer 已通过共享 cursor、同步 drain 和关闭 join 降低有界 ring 中的丢对风险，这不把它升格为永不丢失的内核 tracing API。

12.6 审批、取消与回滚

当前审批 token 是 Host/Guest 产品层协议，有效等待上限为 25 秒，超时或 controller 断开即 deny。它不是内核 V3 grant，也不取代 capability/scope/provenance。已批准的调用只在当前绑定参数与有效期内有效，不提供跨 session 持久授权。

Ctrl-C 是协作式取消当前 turn。它能停止尚未执行的委派或调用，但不能对已经 commit 的文件/消息副作用自动实现事务回滚。因此高风险效果的核心保证是“执行前审批与内核 gate”，不是“执行后一定可撤销”。

12.7 V2、V3 与 Task Channel 不同保证

V2 以 typed request 支持观察后再选工具，但不冻结整张 DAG。启用 AGENT_EXECUTION_CONTRACT_F_ENFORCE 的 V3 才绑定 contract/node/schema/predecessor/attempt/deadline/provenance envelope。不能因为 V2 也经过 capability/provenance 检查，就宣称它具有 V3 frozen guarantee。

Task Channel 当前证明 SQ/CQ、deadline、cancel 和 exactly-one completion。provider 同步且只接受 null input/output NONE，RESOURCE_IMPORT fail closed。它尚未承载 Nexus 业务 payload，不能作为真实文件查询或模型 wire 的异步 backend。

12.8 MCP/A2A 与外部资源

仓库内 MCP/A2A 仍是 deterministic in-memory 对象映射 prototype。它验证 tools/list、schema 和错误对象等映射逻辑，不是真实 MCP stdio/HTTP server、MCP Client、streaming transport、Task Channel adapter 或跨实现互操作证明。Nexus N1、Host model wire 与 MCP wire 是不同层。

本轮黄金场景只使用本地资料。未来 Web Search、论文 API 或完整 MCP 必须作为 Host gateway 的显式受控工具进入 Guest，并将外部输出标记为不可信数据。Host 不得因为“接入方便”而选工具、读业务文件或伪造结果。

12.9 内核和文件系统边界

内核不包含 JSON、模型 SDK、HTTP、TLS 或 MCP wire；这是机制/策略分层，不是待补全的网络栈。当前基于教学 uCore 的单 Hart、进程上限和文件大小上限约束了 Nexus 镜像与长驻能力。我们通过压缩 history 和 image-size gate 适配现有 ABI，不为了演示修改文件系统上限。

Context mirror 固定为 7 页，其中 6 页由内核只读投影，1 页是用户 cache；Context Path 有界且保留 dropped/oldest/latest 信息。metadata catalog 是当前 boot 的显式登记状态，workflow fence receipt 会标记 volatile。这些边界使系统可复验，也要求我们不把它描述成持久全局数据库。

12.10 性能证据的外推边界

one-shot 数据固定在 commit 2b14fb1f74b9bd093e6de939a16554620835699e 及其记录的工具体链、QEMU/Host 环境，适合工程复验，不代表其他硬件、其他负载或长期生产运行。Catalog grid 的每格 15 个 inner pair 来自同一 boot 内重复，不能当成 15 个独立 boot。EEVDF 实验最多覆盖 4 个物理并发 workflow entity，不是 Nexus 四个业务 Agent 的端到端延迟测试。

Core path 的 16/16 获胜不会被扩大成端到端加速；E2E 实际仅 3/16 获胜且中位差为

+13.452 ms。同样，Task 的 sequence 延迟不能单独证明 SQ/CQ 设计的因果优势；它同时包含了当前 provider、队列和实现开销。

本项目的当前承诺

我们承诺的是：在当前 uCore 边界内，Agent 成为内核可识别、可调度、可授权、可记账和可审计的对象；Nexus 交付显式 live 入口与已验证的同路径 replay，并把 Guest 委派、内核工具和双窗口观测连成可交互纵向闭环。我们不承诺通用 Agent 云、完整 MCP 互操作、跨重启持久、任意 shell 或无界长会话。

工程组织、交付与演进

13.1 增量工程原则

我们的实现不另建一套与 uCore 平行的“Agent 内核”，而是把身份、Context、工具、资源、观测和调度嵌入现有 PCB、syscall、VFS、IPC 与 scheduler 路径。业务策略保留在 Guest 用户态，provider 适配保留在 Host，而内核只处理有界二进制 ABI。这一原则使每个新功能都能回答：它改变了哪个既有系统责任，如何由内核强制，又如何被 Guest 消费。

13.2 仓库组织

表 13.1: 主要代码目录与责任

目录	责任	交付证据
os/agent_*.c	内核 identity、Context、工具、resource、metadata、observe、contract、fence	参与生产内核对象链接，受 module/UAPI/stack gate 约束
user/lib/ 与根目录、 user/include/ 中的 UAPI 头	syscall wrapper、ABI 布局和公共常量	kernel/user size/offset 静态断言
user/src/	专项 Guest tests、 labdemo_ucore、 agentlive_ucore、 agentnexus_ucore	真实 RV64 应用与 QEMU markers
user/lib/agent_nexus.c	N1 codec、role-filtered tool view、 artifact 校验与 identity registry	与 Nexus Guest 同步编译链接
host_tools/	CLI、daemon、provider relay、 observer、strict validator	Host unit/mutation tests 和两窗口产品入口
scripts/	边界 checker、stack check、 QEMU runner	fail-closed 的静态和动态 gate
one_shot_metrics/	一次性采集计划、 raw/tables/manifest、重绘和验证	30 boot、33 raw、19 tables、 7,498 rows
docs/agentos/	架构、API、安全、验证、 Console/Nexus 边界	实现与主张对应索引

13.3 四类产品入口

四个入口各自回答不同问题，不能用模型的不确定性替代性能可复现性。

makecontest-demo

在同一 AgentOS Guest 中运行 traversal/compat 与 indexed/native 的固定 AB/BA 工作流，是可比较的竞赛性能基线。

makedual-platform-run

构建并运行 Plain uCore 与 AgentOS-uCore 双目标，用于验证跨目标业务合同和工作量对齐。

makeagent-live-demo

运行单 Agent 的 Guest model/tool loop；默认使用 replay，live provider 需显式选择。

makeagentos-nexus

以 live provider 启动 daemon 与 controller；只读窗口由 makeagentos-nexus-observe 单独启动。

对 Nexus，agentos-nexus-image 只构建镜像，agentos-nexus-check 只运行本地静态/Host 合同，agentos-nexus-replay 启动真实 QEMU strict replay。agentos-nexus 默认选择 DeepSeek/live provider，agentos-nexus-deepseek 是其显式别名；两者都依赖人工提供的网络与 API key，且均需另启 observer 窗口。这些入口名称同时约束证据层次。

13.4 构建与回归

13.4.1 建议顺序

Listing 13.1: 从快速合同到 QEMU 产品的分层验证

```

1  # Host / protocol / static contracts
2  make local-host-selftests
3  make agent-uapi-check TOOLPREFIX=riscv64-linux-gnu-
4  make agent-module-check TOOLPREFIX=riscv64-linux-gnu-
5
6  # Real production objects and stack budgets
7  make build TOOLPREFIX=riscv64-linux-gnu-
8  make kernel-stack-check TOOLPREFIX=riscv64-linux-gnu-
9  make user-stack-check TOOLPREFIX=riscv64-linux-gnu-
10
11 # Product-specific contracts and QEMU replay
12 make agentos-nexus-check
13 make agentos-nexus-image TOOLPREFIX=riscv64-linux-gnu-
```

```
14 make agentos-nexus-replay TOOLPREFIX=riscv64-linux-gnu-
```

不同本地 RISC-V 工具链可能使用 `riscv-none-elf-` 或 `riscv64-linux-gnu-`，实际以当前仓库 `Makefile` 和环境为准。Windows 下的现场产品通常通过 WSL 运行 `make/QEMU`，Host CLI 与 key 路径依照已有入口转换；不将 API key 传入 Guest 镜像。

13.4.2 共享输出约束

Console/Nexus image 复用 `build/kernel` 与 `nfs/fs*.img`，因此不应在同一 `work-tree` 并发构建两个镜像。性能采集还必须避免与 `observer` 或其他 QEMU 实例并发，以免改变 Host 负载和结果口径。一次性数据目录已经写入 `COMPLETED`，日常 CI 不会重跑或覆盖它。

13.5 交付物和可追溯性

竞赛交付不只包含一个可启动镜像，还包含：

- 可从源码重建的 Plain/AgentOS/Nexus Guest 目标；
- UAPI 头文件、生产对象清单与调用图栈 `gate`；
- 确定性 `contest-demo` 与 `strict Nexus replay fixture`；
- Host controller/observer 事件、Guest 串口输出和 `validator` 结果；
- `one-shot raw`、`manifest`、`long tables`、`validation warning` 和可重绘图表；
- 架构、API、安全、验证、Nexus 操作和本决赛报告。

关键主张能够通过“文档章节 → 实现文件 → `checker/QEMU` 入口 → `raw/table/fixture`”回溯。附录A给出高信号源码索引，附录B给出复验步骤与数据解释。

13.6 从确定性流程到 Nexus 的工程演进

第一阶段以 `labdemo_ucore` 建立 `Orchestrator/Sentinel/Investigator/Recovery` 的真实多 Agent 身份、`route`、`Context` 和文件工作流，为 `traversal/indexed` 配对比较提供可重复基线；Plain uCore 与 AgentOS-uCore 的双目标对齐由 `rp_* Guest` 和 `dual-platform-run` 另行验证。第二阶段的 `agentlive_ucore` 建立 `model request/response`、真实 `typed V2` 工具和结果回灌的闭环。第三阶段将该循环扩展为长驻 CLI，再用 Nexus 统一动态委派、`artifact` 和独立 `observer`。

这条演进线保留了旧路径的用途：`contest-demo` 继续是性能基准，不被模型不确定性替代；`agent-live-demo` 仍是更小的 `model-loop` 验收；Nexus 才是现场交互主演示。

13.7 后续演进路径

下列方向是建立在当前边界上的工程计划，不是已完成能力：

1. 为 artifact store 增加内核或可验签名的 object ACL/seal，并设计跨 boot 恢复与容量回收；
2. 让 Task Channel 承载有 generation-safe handle 的真实业务 input/result，保留 cancellation 与 exactly-one completion；
3. 将 Guest 调用级审批升级为内核可验证的有界 token，同时保留人机交互体验；
4. 为外部资源增加 Host gateway 工具，将网络输出统一标记为不可信，不让 Host 获得业务决策权；
5. 在保持 Nexus N1 和内核 ABI 分层的前提下，实现真实 MCP stdio/HTTP Client/Server 互操作；
6. 在多 Hart 与更大工作流上扩展 EEVDF、观测和资源账户实验，增加统计独立 boot 而不复制 inner pair。

每一步都应先定义责任边界与可执行验收，再增加 API。我们不为了展示丰富度而增加没有真实 consumer 的内核接口。

总结

我们在 uCore 上回答了一个具体的系统问题：当 Agent 从普通用户程序变成会长时间等待、动态调用工具、跨身份传递数据并消耗多类资源的执行体时，操作系统应该为它提供什么机制。

我们的答案是分层治理：Host 负责 TLS/provider，Guest runtime 负责历史、规划和工具选择，内核负责身份、lifecycle、scope、capability、provenance、结构化参数、等待唤醒、资源账户、调度与审计。LLM SDK、JSON、HTTP 与 MCP wire 保留在 Guest/Host 层。

六项赛题任务沿一条系统路径衔接。内核身份为 Context 和 capability 提供主体；typed V2/V3 将工具调用变成可拒绝的系统交互；Context、provenance、Evidence 和 fence 把运行结果连成有界证据；metadata/index/watch 让文件查询真正走内核数据路径；event 与 workflow EEVDF 使 Agent 等待时不忙轮询，也不能通过增加线程放大份额；最后，确定性 contest-demo 与交互式 Nexus 分别承担性能复现与产品演示。

Nexus 是本项目的纵向收口。用户在左窗口连续提交目标，Coordinator 根据真实结果委派 System、Research 或 Analyst；三个 worker 拥有独立内核身份和 Context，并共享 workflow 资源域与调度实体；右窗口以可读的 task/correlation/Context 展示内核实际允许、拒绝、挂起和记录的事实。Host 只做传输与呈现，不冒充业务 Agent。固定 replay 使这条路径在无网时仍可复验，而 live provider 只在真实 API 闭环完成时才作相应披露。

实验结果也保留了必要的负结论。16 个 paired boot 中，indexed core 在 16/16 配对中更快，中位加速 3.118 $\times$ ；但端到端仅 3/16 更快，中位差为 +13.452 ms。这说明一个快的内核查询路径不会自动变成更快的整体产品。我们将 core 与 E2E 分开报告，将 inner pair 与 independent boot 分开解释，并保留 EEVDF 串口拼接 warning，因为可诚实质疑的证据比看起来完美的数字更有价值。

当前产品仍有清晰限制：四业务角色、有界 session/artifact、本地资料、Guest 审批、boot-volatile store、Task Channel null provider 和 MCP/A2A in-memory prototype。我们没有用“将来可以扩展”把它们改写成已完成能力。这些边界反而为后续演进提供了可执行路线：先升级 artifact/Task/approval 的可验边界，再引入外部资源与互操作，始终不把业务策略推进内核。

最终结论

AgentOS-uCore 将 Agent 行为纳入可治理的系统工作负载：可识别、可授权、可挂起、可记账、可调度、可追踪，并且在拒绝时仍能返回结构化观察供策略重新规划。

关键源码与证据索引

本附录提供“主张到实现”的高信号导航。行号会随合法演进变化，因此以文件和稳定符号为主，不把某一次快照行号当成 ABI。

A.1 内核身份、生命周期与 Context

表 A.1: 任务一与 Context 主路径

文件	主要责任	检查要点
os/agent_identity.c	trusted exec、role/capability 与 identity 安装	普通进程不能自报成 Agent；role 对 capability 映射不因 Nexus 放宽
os/workflow_lifecycle.c	workflow create/member/-closing/generation	活跃成员、关闭门禁与 reuse 安全
os/agent_core.c	syscall 核心调度、identity/scope 共享校验	公共 gate 位于业务工具之前
os/agent_context.c	Context 槽与 mirror 更新	6 页可信只读 mirror 和 1 页用户 cache 分离
os/agent_context_path.c	Path append/query/rollback/snapshot	sequence、oldest/latest、dropped 与 provenance 语义
根目录 agent*_abi.h、 os/agent.h 与 user/include/agent.h	用户/内核 ABI	size/offset/static assert 是否一致

A.2 工具、合同与 Task Channel

表 A.2: 任务二主路径

文件	主要责任	检查要点
os/agent_tool_protocol.c	25 工具目录、name/ID、typed KV 解析与 response	未知工具、重复 key、类型/长度错误先拒绝
os/agent_execution_contract.c	V3 frozen node、edge、schema、attempt、deadline 与 provenance	只有 ENFORCE 模式声称 contract-bound guarantee
os/agent_identity_lease.c	Phase Lease 与有界授权	tool、effect、provenance、deadline 和 resource 同时绑定
os/agent_task_channel.c	SQ/CQ、deadline、cancel、exactly-one completion	当前 provider 仅 null input/output NONE
os/agent_task_bridge.c	Task Channel 与现有执行路径的窄桥接	RESOURCE_IMPORT 应 fail closed
user/src/agent_toolabi_ucore.c	V2/V3/ABI Guest 动态检查	真实 RV64 参数与错误码
user/src/agent_task_ucore.c	compact batch/scalar V3/SQ-CQ 专项路径	16-op sequence 的工作量一致性

A.3 Context、Provenance、Evidence 与 Fence

表 A.3: 任务三与截面证据主路径

文件	主要责任	检查要点
os/agent_provenance.c	provenance 合法性、传播与 mask gate	不可信标签不得在跨 Agent 流转中被洗白
os/agent_evidence_ring.c	ordinary/critical 有界 ring、reserve/-commit/seal	BUSY 槽隐藏，critical 不被 ordinary 挤占

文件	主要责任	检查要点
os/agent_workflow_fence.c	lifecycle cut 和 receipt	metadata/live query、VFS、credit、evidence 的固定顺序
scripts/test-agent-evidence-ring.py	ring	gap、rollover、previous
scripts/check-workflow-fence.py	model/mutation	root/challenge/credit digest
scripts/test-workflow-fence.py	生产调用链静态 gate	无隐式 fence 或旁路发布
scripts/test-workflow-fence.py	fence 功能模型	idempotent retry、conflict、stale、copyout retry cache

A.4 文件查询、索引与 Live Watch

表 A.4: 任务四主路径

文件	主要责任	检查要点
os/agent_metadata_catalog.c	boot-volatile 显式 metadata catalog	普通 create 不自动进 catalog
os/agent_metadata_query.c	scan/index 查询与工作量计数	同一 predicate 结果一致，不仅比较时间
os/agent_metadata_directory.c	目录/对象记录与 generation	inode incarnation 不与旧文件混淆
os/agent_live_query_events.c	ENTER、UPDATE、LEAVE、pending 与 resync generation	queue 丢失后 ACK 不能清除未处理 gap
user/src/agent_eval_ucore.c	catalog grid 实验 Guest	24/64/96 × 1/2/4/8，每格 15 inner pair
scripts/check-agent-live-query-fs.py	VFS/live-query 静态边界	不回退到文本路径或持久 journal

A.5 等待、IPC、资源与 EEVDF

表 A.5: 任务五与跨模块调度主路径

文件	主要责任	检查要点
os/agent_ipc.c	MESSAGE route/watch/wait- /wakeup 和有界 source queue	require-watch、scope、provenance、 queue-full 错误
os/agent_backg round.c	后台维护 pending flag	原子发布、查询和消费
os/resource_co ntroller.c	U/P/F 账户、 vector reserve/com- mit/cancel/release	$H = U + P + F$ ，失败无部分提交
os/proc.c 与 os/workflow_sc heduler.c	workflow EEVDF 选择与成员折叠	线程数不放大 workflow 份额
user/src/agent loop_ucore.c	等待/唤 醒/heartbeat Guest 验收	等待期间不忙轮询
user/src/agent _eevdf_ucore.c	EEVDF 实验 Guest	exact ready-age、service cycles 与 Jain 计算所需数据
scripts/test-w orkflow-credi t-domain.py	U/P/F mutation model	hard limit 时机、trim F、fence exact U

A.6 综合工作流与 Nexus

表 A.6: 任务六和交互产品主路径

文件	主要责任	检查要点
user/src/labde mo_ucore.c	确定性多 Agent 科研恢复工作流	traversal/indexed 结果等价与 AB/BA 顺序平衡
baseline_ucore /user/src/rp_*. c 与 user/src/r p_*.c	Plain uCore/AgentOS- uCore 双目标 Guest	配合 scripts/run-dual-pla tforms.sh 与 host_tools/compare_dual_ platform_state.py 验证业务 合同和工作量对齐
user/src/agent live_ucore.c	单 Agent 长驻 model/tool loop	Guest 保有 history/catalog/decision/result feedback

文件	主要责任	检查要点
user/src/agent_nexus_ucore.c	四角色长驻会话、动态委派、observer 和审批	System no-input、Research fail/replan、Analyst report、deny zero effect
user/include/agent_nexus_protocol.h	N1 envelope、artifact handle/-manifest/capsule ABI	canonical 44-byte wire、state/flag/handle 布局
user/lib/agent_nexus.c	协议 codec、role-filtered tool、artifact 发布/验证	identity registry、owned/brokered 规则和 full SHA-256
user/include/agentnexus_seed.h	版本化本地 case/measurement/state capsule	source/derived 字段与 historical/current 范围
host_tools/agentos_relayd.py	串口帧、provider、controller/observer 分流	Host 不读 Guest 业务文件，schema 严格
host_tools/agentos_cli.py	交互输入、slash command、审批	controller 单 owner，Ctrl-C 只取消 turn
host_tools/agentos_observe.py	只读 telemetry 视图	task、correlation、Context、audit 与 scheduler 高信号字段
host_tools/validate_agentos_nexus_replay.py	strict replay oracle	request digest、因果顺序、artifact/provenance、audit pair 和 close 顺序

A.7 性能数据和图表

表 A.7: 性能证据主路径

文件/目录	主要责任	检查要点
one_shot_metrics/CANONICAL_RUN	指向规范数据集	正文不从临时目录取数
one_shot_metrics/data/20260811/raw/	33 个原始串口文件	结果表可追溯到 raw 前缀与 SHA-256

文件/目录	主要责任	检查要点
one_shot_metrics/data/20260811/tables/	19 张长表	boot/sample/pair/round/workflow 标识不丢失
one_shot_metrics/data/20260811/manifest.json	提交、Guest 变体、负载、工具版本和 inventory	规范运行不被后来重绘覆盖
one_shot_metrics/data/20260811/validation.json	数据完整性与 warning	EEVDF boot 05 拼接记录保留
one_shot_metrics/plot.py	从长表重绘图件	不启动 QEMU，不从文档聚合值反推
one_shot_metrics/validate.py	离线完整性验证	输出到仓库外 scratch，不修改 canonical run

复验指南与附加数据解释

B.1 环境准备

以当前仓库根目录为工作目录，准备 RV64 交叉工具链、QEMU、Python 3 与项目 Makefile 需要的基础命令。首先记录代码 commit、工具链版本、QEMU 版本、Host OS/CPU 和命令行参数。如果使用 Windows，建议在 WSL 中运行 make/QEMU，并保证同一 worktree 同时只有一个写共享 image 的构建。

仓库外 API key 只供 live provider 入口使用。agentos-nexus-replay 不读 key，也不联网。复验时不应将 key 打印到命令、串口日志或报告中。

B.2 分层构建

Listing B.1: 建议的静态、编译与 Guest 回归顺序

```
1 # 1. Fast local contracts
2 make local-host-selftests
3 make agent-uapi-check TOOLPREFIX=riscv64-linux-gnu-
4
5 # 2. Production module and stack gates
6 make agent-module-check TOOLPREFIX=riscv64-linux-gnu-
7 make build TOOLPREFIX=riscv64-linux-gnu-
8 make kernel-stack-check TOOLPREFIX=riscv64-linux-gnu-
9 make user-stack-check TOOLPREFIX=riscv64-linux-gnu-
10
11 # 3. Guest functional suite
12 make agentos-test TOOLPREFIX=riscv64-linux-gnu-
```

快速 checker 失败时先修复合同，不要直接启动 QEMU 观察“是否刚好通过”。构建通过后仍不能声称功能通过；必须运行对应 Guest 场景并检查负路径 marker。

B.3 Nexus 产品复验

B.3.1 静态合同

```
1 make agentos-nexus-check
```

```
2 make agentos-nexus-image TOOLPREFIX=riscv64-linux-gnu-
```

第一条命令检查 Host profile/schema、N1 Guest loop、artifact/provenance、审批和 observer 来源边界，不启动 QEMU。第二条命令构建 RV64 Guest 与镜像，并在 mkfs 前检查 Nexus raw binary 不超过 274432 bytes。

B.3.2 固定 replay 闭环

```
1 make agentos-nexus-replay TOOLPREFIX=riscv64-linux-gnu-
```

成功不只是 Guest 打印 passed。应同时保留 controller、observer 和 daemon 输出，并确认：

- HELLO 的 profile=nexus 与 features=task_event_v1 正确；
- 四个业务 Agent 都拥有正数且唯一的 control identity；
- System 完成无输入快照并发布 artifact；
- Research 先有一个 FAILED terminal，后有新 task 成功，失败 task 不被伪改为成功；
- Analyst report 同时绑定 System/Research handle 与 payload SHA-256；
- publish_report 的审批决定字段与请求完全对齐，deny 后内核 artifact_update 零执行；
- observer 中 TASK/audit/correlation/Context 可对齐，正常关闭后无新 telemetry。

对 digest-bound fixture 进行修改时，必须从当前 Guest 真实 MODEL_REQUEST 捕获新 digest，不得使用通配符、空 fixture 或 Host 归一化掩盖请求漂移。修改 history/model projection 后，至少连续两个 fresh strict boot 通过，才能说明该 fixture 对当前源码可复现。

B.3.3 现场交互

```
1 # Window 1: daemon + controller
2 make agentos-nexus TOOLPREFIX=riscv64-linux-gnu-
3
4 # Window 2: read-only observer
5 make agentos-nexus-observe
```

左窗口提交用户目标并处理审批，右窗口展示内核事实。不要同时启动第二个会写共享 image/runtime 的 Nexus daemon。使用 /quit 正常关闭，不删除 runtime 文件伪装会话结束。

B.4 Contest demo 与 paired 证据

```
1 make contest-demo-check
2 make contest-demo TOOLPREFIX=riscv64-linux-gnu-
```

采集时必须保留同一 AgentOS Guest 的 traversal/indexed 输出、比较摘要、负载计数、target/order/seed 和 Host 环境。单次 AB 或 BA 不能提供顺序平衡；对外数字应以 paired boot 为单位。Plain uCore 与 AgentOS-uCore 的双目标比较由 dual-platform-run 负责，交互式 Nexus 也不能代替这个确定性基准。

B.5 一次性数据集的离线检查

```
1 python one_shot_metrics/validate.py \
2   --tables one_shot_metrics/data/20260811/tables \
3   --output ../agentos-20260811-reproduced/validation.json
4
5 python one_shot_metrics/plot.py \
6   --tables one_shot_metrics/data/20260811/tables \
7   --output-dir ../agentos-20260811-reproduced/figures \
8   --format png,pdf
```

两条命令只做离线验证和重绘，不启动 QEMU。输出应放到仓库外 scratch，不覆盖 one_shot_metrics/data/20260811。规范目录包含 COMPLETED 和 145-entry manifest inventory，日常开发不会重新采集它。

B.6 附加 I/O 混合 scope 图

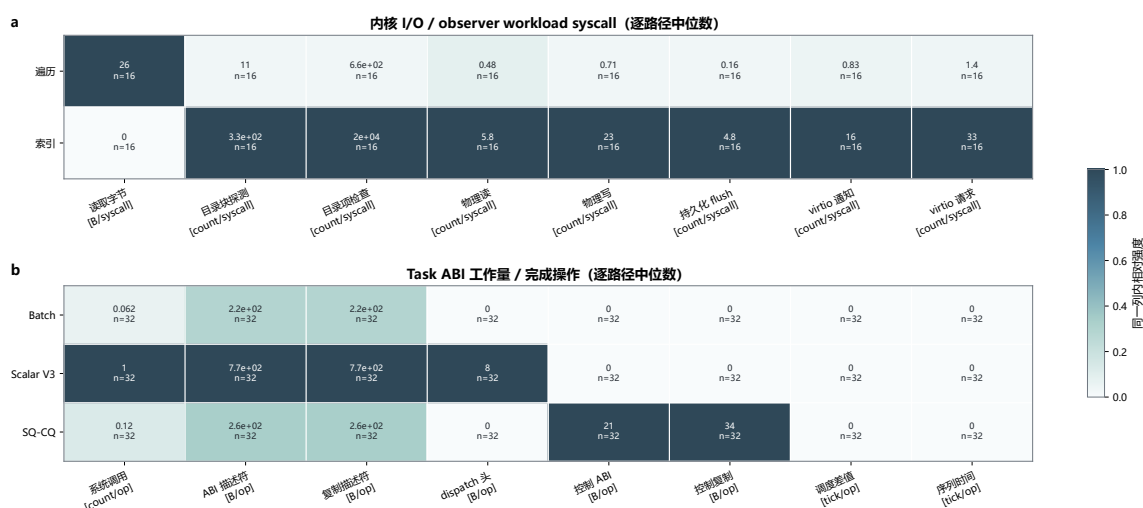


图 B.1: 内核 I/O 和工作量归一化热力图；owner/window/scope 不同，仅作附加工程视图

该图刻意放在附录。bytes_read 是 workflow lane/core 报告的 total，其他 contest numerator 主要是 shared global/end-to-end delta，Task 则是直接 work count/bytes。它们的

owner、window 和 denominator 不同，不能混成单一进程 I/O 归因。热力图用于快速查找工作量差异，不是主性能结论。

B.7 核心数字复核表

表 B.1: 正文引用的核心数字及其正确解释

项目	规范值	解释约束
配对 workflow core 中位时间	16 fresh QEMU boot 34.713/13.294 ms	boot 是独立单位，AB/BA 各占一半 traversal/indexed core，不是纯 syscall latency
core 配对加速 E2E 配对	3.118 $\times$ ，16/16 wins 3/16 wins，+13.452 ms	仅对当前 core 窗口 positive 表示 indexed 窗口更慢
AB/BA 顺序分层 catalog grid	1.4535 $\times$ /5.4805 $\times$ 3 $\times$ 4 $\times$ 15 = 180 pair	提示强顺序/boot 层影响 15 是同 boot inner pair，不是 15 boot
Task sequences Task 中位数	96，4 boot $\times$ 8 round 561/2051/1620.5 us	每序列 16 op compact batch/scalar V3/SQ-CQ 完整 sequence
EEVDF wake	504 probe；425 zero、 79 one tick	exact trace，不从直方图插值
Jain 中位数	1.000000，0.999985， 0.999993，0.999985	并发 1/2/3/4；1 是平凡基线
数据集规模	30 boot, 33 raw, 19 tables, 7,498 rows	一次性工程采集，不是跨硬件总体推断

B.8 报告工程的重建

```
1 cd docs/final-report
2 latexmk -xelatex -interaction=nonstopmode -halt-on-error main.tex
3 cp main.pdf AgentOS-uCore-final-report.pdf
```

若环境中的 latexmk 不可用，依次执行一次 XeLaTeX、一次 BibTeX 和两次 XeLaTeX，再复制最终命名 PDF，生成结果等价。

构建后应检查：所有图文件存在；无 undefined reference/citation；无 overfull box；所有页面在 A4 范围内；图题、表题、页眉和页码不重叠；连续页面无空白占位图。最终 PDF 固定输出为 docs/final-report/AgentOS-uCore-final-report.pdf。

参考文献

- [1] LearningOS Community. ucore-tutorial-code-2025s. <https://github.com/LearningOS/uCore-Tutorial-Code-2025S>, 2025. AgentOS-uCore 的教学内核基础; 访问日期: 2026-08-11.
- [2] Linux Kernel Documentation. Cpu accounting controller. <https://docs.kernel.org/6.11/admin-guide/cgroup-v1/cpuacct.html>, 2024. 访问日期: 2026-08-11.
- [3] Linux Kernel Project. rstat.c. <https://github.com/torvalds/linux/blob/master/kernel/cgroup/rstat.c>, 2026. 访问日期: 2026-08-11.
- [4] Linux Kernel Documentation. Bpf ring buffer. <https://docs.kernel.org/6.6/bpf/ringbuf.html>, 2023. 访问日期: 2026-08-11.
- [5] Linux Kernel Documentation. Eevdf scheduler. <https://docs.kernel.org/scheduler/sched-eevdf.html>, 2026. 访问日期: 2026-08-11.
- [6] Jens Axboe. Efficient io with io_uring. https://kernel.dk/io_uring.pdf, 2019. 访问日期: 2026-08-11.
- [7] Haiku Project. Practical file system design with the be file system. <https://www.haiku-os.org/legacy-docs/practical-file-system-design.pdf>, 1999. 访问日期: 2026-08-11.
- [8] Anthropic. How tool use works. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works>, 2026. 访问日期: 2026-08-11.
- [9] Anthropic. How claude code works: The agentic loop. <https://code.claude.com/docs/en/how-claude-code-works>, 2026. 访问日期: 2026-08-11.
- [10] Model Context Protocol. Mcp specification 2026-07-28. <https://modelcontextprotocol.io/specification/2026-07-28>, 2026. 访问日期: 2026-08-11.
- [11] A2A Project. A2a protocol specification v1. <https://a2a-protocol.org/latest/specification/>, 2026. 访问日期: 2026-08-11.